The sdev Book

A Complete Guide to the sdev Programming Language

First Edition · 2026

Table of Contents

Table of Contents 3

The sdev Book 23

Table of Contents 23

1. Introduction 28

1.1 Why sdev? 28

1.2 A Brief History 29

1.3 Who Should Read This Book 29

1.4 How To Use This Book 30

1.5 Conventions 30

1.6 A First Taste 31

2. Getting Started 31

2.1 Installing sdev 31

2.2 Your First Program 32

2.3 The REPL 32

2.4 Project Structure 33

2.5 Running From the IDE 33

2.6 Anatomy of a Program 33

2.7 Editor Setup 34

2.8 Common Pitfalls For Beginners 34

3. Variables and Values 34

3.1 Forging Variables 35
3.2 The Six Primitive Types 35
3.3 Type Inspection 35
3.4 Type Conversion (morph) 36
3.5 Constants (eternal) 36
3.6 Naming Rules 37
3.7 Scope 37
3.8 Nothing, Truthiness, Equality 37
3.9 Numbers In Detail 37
3.10 Strings In Detail 38
4. Operators and Expressions 38
4.1 Arithmetic 38
4.2 Comparison 38
4.3 Logical 39
4.4 String Operators 39
4.5 Membership 39
4.6 Pipelines (|>) 39
4.7 Operator Precedence 39
4.8 Truthy Conversions 40
5. Control Flow 40
5.1 If / Then / Else 40
5.2 When (Pattern Match) 40
5.3 While And Until 41
5.4 For ... In 41

5.5 Escape And Skip 41
5.6 Labelled Loops 42
5.7 Match (Destructuring) 42
6. Functions 42
6.1 Defining a Function 42
6.2 Default and Keyword Arguments 43
6.3 Variadic Arguments 43
6.4 Returning Values 43
6.5 Anonymous Functions (Lambdas) 43
6.6 Closures 43
6.7 Higher-Order Functions 44
6.8 Recursion 44
6.9 Generators (yield) 44
6.10 Decorators 45
7. Collections 45
7.1 Lists 45
7.2 Dictionaries 45
7.3 Slicing 46
7.4 Map / Filter / Reduce 46
7.5 Sets 46
7.6 Tuples 46
7.7 Iteration Idioms 46
7.8 Performance Notes 47

8. Strings In Depth 47
8.1 Literals 47
8.2 Indexing And Slicing 47
8.3 Searching 47
8.4 Manipulation 48
8.5 Encoding 48
8.6 Formatting 48
8.7 Regular Expressions 48
8.8 Unicode And Locales 48
9. Object-Oriented Programming 49
9.1 Defining a Class 49
9.2 Creating Instances 49
9.3 Inheritance 49
9.4 Class Methods And Fields 50
9.5 Mixins (weave) 50
9.6 Operator Overloading 50
9.7 Properties 50
9.8 Abstract Classes And Interfaces 51
10. Error Handling 51
10.1 Raising Errors 51
10.2 Attempt / Rescue / Finally 51
10.3 Error Types 51
10.4 Multiple Rescues 52

10.5 Re-raising 52
10.6 Stack Traces 52
10.7 Assertions 52
10.8 Defensive Programming Idioms 53
11. Concurrency 53
11.1 Async Functions 53
11.2 Dispatch And Await 53
11.3 Parallel Composition 54
11.4 Channels 54
11.5 Cancellation 54
11.6 Time And Delay 54
11.7 The Kernel 54
12. Modules and Libraries 55
12.1 Local Modules 55
12.2 Gist Packages 55
12.3 Versioning 55
12.4 Manifests 55
12.5 Authoring A Gist Module 56
12.6 Module Scope 56
12.7 Dynamic Loading 56
13. The UI Toolkit 56
13.1 Windows 56
13.2 Headings, Labels, Paragraphs 57

13.3 Buttons 57
13.4 Inputs 57
13.5 Textareas 57
13.6 Checkboxes And Radios 57
13.7 Sliders And Selects 58
13.8 Tables 58
13.9 Tabs 58
13.10 Menus 58
13.11 Layout: Rows, Columns, Groups 59
13.12 Reactive Values 59
13.13 Event Handlers 59
13.14 Customisation 59
13.15 A Complete Example 59
14. Graphics and Canvas 60
14.1 Opening A Canvas 60
14.2 Drawing Primitives 60
14.3 Colors 61
14.4 Animation 61
14.5 Images And Sprites 61
14.6 Transforms 61
14.7 Mouse And Keyboard 61
14.8 Bouncing Ball Example 61
15. Maps and GIS 62
15.1 Opening A Map 62

15.2 Markers And Popups 62
15.3 Tile Layers 62
15.4 Polylines And Polygons 62
15.5 GeoJSON 63
15.6 Heatmaps And Clustering 63
15.7 Events 63
15.8 Routing And Distance 63
15.9 A Complete Map Example 63
16. The Standard Library 64
16.1 Built-in Function Table 64
16.2 I/O 67
16.3 Math 67
16.4 Collections 67
16.5 Strings 67
16.6 Time 67
16.7 Network 67
16.8 System 68
16.9 Encoding 68
16.10 Cryptography 68
17. Tooling 68
17.1 The Web IDE 68
17.2 VS Code Extension 69
17.3 CLI Reference 69
17.4 Bytecode Compiler 69

17.5 The sdev Assistant 69
17.6 Linter 69
17.7 Formatter 70
17.8 Debugger 70
17.9 Test Runner 70
18. Advanced Topics 70
18.1 JavaScript Interop 70
18.2 Matrix Math 71
18.3 Buffers and Pointers 71
18.4 The Virtual Kernel 71
18.5 Garbage Collection 71
18.6 The Bytecode VM 72
18.7 Multi-Language Source 72
18.8 Embedding sdev 72
18.9 FFI 72
19. Performance 72
19.1 Choose The Right Runtime 73
19.2 Profile Before Optimising 73
19.3 Avoid Quadratic Patterns 73
19.4 Cache Computations 73
19.5 Reuse Buffers 73
19.6 Compile To Bytecode 74
20. Best Practices 74

20.1 Name Things Well 74
20.2 Prefer Verbs 74
20.3 Keep Functions Small 74
20.4 Embrace Immutability 74
20.5 Document Intent 75
20.6 Test Early, Test Often 75
20.7 Fail Loudly 75
20.8 Prefer Composition Over Inheritance 75
21. A Cookbook of Examples 75
21.1 FizzBuzz 76
21.2 Word Count 76
21.3 Frequency Histogram 76
21.4 Todo App UI 76
21.5 HTTP Fetch 77
21.6 Bouncing Ball Animation 77
21.7 Palindrome Checker 77
21.8 Fibonacci (Memoised) 78
21.9 Calculator UI 78
21.10 Web Scraper 78
21.11 Map Of Capitals 79
22. Building A Real Application 79
22.1 Project Layout 79
22.2 Storage Module 79
22.3 Render Module 80

22.4 UI 80
22.5 Wiring It Up 80
22.6 Adding Cloud Sync 80
22.7 Publishing 80
23. Frequently Asked Questions 81
23.1 Is sdev fast? 81
23.2 Can I use it in production? 81
23.3 Why not just use Python? 81
23.4 Why two runtimes? 82
23.5 Where do I report bugs? 82
23.6 Can I contribute? 82
23.7 How do I keep my files between sessions? . . . 82
23.8 Why "forge" instead of "let"? 82
24. Glossary 83
24.1 A–F 83
24.2 G–N 83
24.3 O–Z 83
25. Design Notes 84
25.1 Verbs Over Symbols 84
25.2 Why end And Not Indentation 84
25.3 Two Runtimes, One Language 84
25.4 Why A UI Toolkit Is Built In 85
25.5 Multi-Language Source 85

25.6 Inspirations 85
26. Appendices 85
26.1 A. Keyword Reference 85
26.2 B. Operator Reference 86
26.3 C. Built-in Function Index 86
26.4 D. Source Languages 86
26.5 E. Bytecode Opcodes 86
26.6 F. Syscall Table 86
26.7 G. Error Code Index 87
26.8 H. CLI Flag Reference 87
26.9 I. Index of Examples 87

sdev Programming Language 89

Complete Documentation, Tutorial & Reference Guide 89
Table of Contents 89
Introduction 90
Getting Started 91
Running in the IDE 91
Running in the Playground 91
Downloading the Desktop App 92
Your First Program 92
Syntax Overview 93
Comments 93
Blocks 93

No Semicolons Required 94
Indentation 94
Variables & Constants 94
Declaring Variables 94
Reassigning Variables 95
Compound Assignment (shorthand patterns) 95
Built-in Constants 95
Multiple Variables 95
Data Types 96
Numbers 96
Text (Strings) 97
Truth (Booleans) 97
Void (Null) 98
Lists 98
Tomes (Dictionaries) 98
Operators 99
Arithmetic Operators 99
Comparison Operators 99
Logical Operators 100
String & List Concatenation 100
Pipe Operator |> 100
Ternary Operator ~ 101
Operator Precedence (high to low) 101

String Operations 102
Control Flow 103
If / Else — ponder / otherwise 103
While Loop — cycle 104
For-Each Loop — iterate through 104
For-In Loop — within 105
Loop Control 105
Nested Loops 106
Functions 106
Basic Functions — conjure 106
Return Values — yield 106
Default Parameters 107
Lambda Functions (Arrow Syntax) 107
Closures 108
Recursive Functions 109
Higher-Order Functions as Parameters 109
Variadic-style Functions 110
Object-Oriented Programming 110
Defining a Class — essence 110
Creating Instances — new 111
Inheritance — extend 111
Multi-level Inheritance 113
Properties and Computed Properties 114

Error Handling 115
Try / Rescue — attempt / rescue 115
Nested Error Handling 116
Assertion 116
Async & Concurrency 117
Async Functions 117
Await 117
Spawn (Concurrent Execution) 117
Delay 118
Built-in Output Functions 118
Built-in Math Functions 119
Core Math 119
Trigonometry 119
Logarithms & Exponentials 120
Advanced Math 120
Random 120
Built-in String Functions 121
Built-in List Functions 122
Built-in Tome (Dict) Functions 123
Input / Output 123
input(prompt?) — Read User Input 123
print(...) / println(...) — Print (Aliases) 124
Character & Code Point Functions 124

Number Base Conversion 125
Number Formatting & Checking 125
String Checking Functions 126
Regex / Pattern Matching 126
Bitwise Operations 127
Base64 Encoding 128
Hash Function 128
Time & Date 128
Functional Programming 129
Buffer / Memory Operations 130
buffer(size) — Create Byte Buffer 130
Buffer Methods 131
pointer(buffer, offset) — Memory Pointer 131
Pointer Methods 131
Error Handling & Control Flow 132
Additional Aliases 132
Higher-Order Functions 133
each — Transform (Map) 133
sift — Filter 133
fold — Reduce 133
seek — Find First 134
every — All Match 134
some — Any Match 135

enumerate — Index + Value 135
zip — Pair Two Lists 135
Type System & Conversion 135
Getting Type — essence() 136
Converting Types — morph() 136
Type Checking Helpers 136
Collections 136
Set 136
Map 137
Queue (FIFO) 138
Stack (LIFO) 138
LinkedList 139
Matrix Operations 140
Creating Matrices 140
Matrix Arithmetic 140
Vector Operations 141
Matrix Utilities 141
Neural Network Example 142
Graphics & Game Development 142
Canvas Setup 143
Fill and Stroke 143
Basic Shapes 143
Text Drawing 144

Gradients 145
Shadows 145
Transformations 145
Path Drawing 145
Turtle Graphics 146
Sprites & Game Objects 148
Color Functions 148
Graphics Math 149
Complete 2D Graphics Example — Starfield 149
File I/O 150
Reading Files 150
Writing Files 151
Append to File 151
File Utilities 151
Networking 152
HTTP GET 152
HTTP POST 152
Examples & Recipes 152
FizzBuzz 152
Fibonacci (Memoized) 153
Binary Search 153
Quicksort 154
Stack-Based Calculator 154

Caesar Cipher 155
Word Frequency Counter 156
Prime Sieve 156
Linked-List Graph Traversal (BFS) 157
Turtle Mandala 158
JavaScript Interop (JS Interpreter Only) 159
Three Syntax Forms 159
Basic Examples 159
Object Literals 160
Arrow Functions 160
Multi-line JS Blocks 161
Leaflet Map Integration 161
Complete Reference Card 162
Keywords 162
Special Symbols 162
Built-in Constants 163
Quick Reference Examples 163

CHAPTER 01

The sdev Book

The sdev Book

The comprehensive reference for the sdev programming language.

Table of Contents

1. Introduction

- 1.1 Why sdev? - 1.2 A Brief History - 1.3 Who Should Read This Book - 1.4 How To Use This Book - 1.5 Conventions - 1.6 A First Taste

2. Getting Started

- 2.1 Installing sdev - 2.2 Your First Program - 2.3 The REPL - 2.4 Project Structure - 2.5 Running From the IDE - 2.6 Anatomy of a Program - 2.7 Editor Setup - 2.8 Common Pitfalls For Beginners

3. Variables and Values

- 3.1 Forging Variables - 3.2 The Six Primitive Types - 3.3 Type Inspection - 3.4 Type Conversion (□CODE0□) - 3.5 Constants (□CODE1□) - 3.6 Naming Rules - 3.7 Scope - 3.8 Nothing, Truthiness, Equality - 3.9 Numbers In Detail - 3.10 Strings In Detail

4. Operators and Expressions

- 4.1 Arithmetic - 4.2 Comparison - 4.3 Logical - 4.4 String Operators - 4.5 Membership - 4.6 Pipelines (□CODE0□) - 4.7 Operator Precedence - 4.8 Truthy

Conversions

5. Control Flow

- 5.1 If / Then / Else - 5.2 When (Pattern Match) - 5.3 While And Until - 5.4 For ... In - 5.5 Escape And Skip - 5.6 Labelled Loops - 5.7 Match (Destructuring)

6. Functions

- 6.1 Defining a Function - 6.2 Default and Keyword Arguments - 6.3 Variadic Arguments - 6.4 Returning Values - 6.5 Anonymous Functions (Lambdas) - 6.6 Closures - 6.7 Higher-Order Functions - 6.8 Recursion - 6.9 Generators (□CODE0□) - 6.10 Decorators

7. Collections

- 7.1 Lists - 7.2 Dictionaries - 7.3 Slicing - 7.4 Map / Filter / Reduce - 7.5 Sets - 7.6 Tuples - 7.7 Iteration Idioms - 7.8 Performance Notes

8. Strings In Depth

- 8.1 Literals - 8.2 Indexing And Slicing - 8.3 Searching - 8.4 Manipulation - 8.5 Encoding - 8.6 Formatting - 8.7 Regular Expressions - 8.8 Unicode And Locales

9. Object-Oriented Programming

- 9.1 Defining a Class - 9.2 Creating Instances - 9.3 Inheritance - 9.4 Class Methods And Fields - 9.5 Mixins (□CODE0□) - 9.6 Operator Overloading - 9.7 Properties - 9.8 Abstract Classes And Interfaces

10. Error Handling

- 10.1 Raising Errors - 10.2 Attempt / Rescue / Finally - 10.3 Error Types - 10.4 Multiple Rescues - 10.5

Re-raising - 10.6 Stack Traces - 10.7 Assertions - 10.8 Defensive Programming Idioms

11. Concurrency

- 11.1 Async Functions - 11.2 Dispatch And Await - 11.3 Parallel Composition - 11.4 Channels - 11.5 Cancellation - 11.6 Time And Delay - 11.7 The Kernel

12. Modules and Libraries

- 12.1 Local Modules - 12.2 Gist Packages - 12.3 Versioning - 12.4 Manifests - 12.5 Authoring A Gist Module - 12.6 Module Scope - 12.7 Dynamic Loading

13. The UI Toolkit

- 13.1 Windows - 13.2 Headings, Labels, Paragraphs - 13.3 Buttons - 13.4 Inputs - 13.5 Textareas - 13.6 Checkboxes And Radios - 13.7 Sliders And Selects - 13.8 Tables - 13.9 Tabs - 13.10 Menus - 13.11 Layout: Rows, Columns, Groups - 13.12 Reactive Values - 13.13 Event Handlers - 13.14 Customisation - 13.15 A Complete Example

14. Graphics and Canvas

- 14.1 Opening A Canvas - 14.2 Drawing Primitives - 14.3 Colors - 14.4 Animation - 14.5 Images And Sprites - 14.6 Transforms - 14.7 Mouse And Keyboard - 14.8 Bouncing Ball Example

15. Maps and GIS

- 15.1 Opening A Map - 15.2 Markers And Popups - 15.3 Tile Layers - 15.4 Polylines And Polygons - 15.5 GeoJSON - 15.6 Heatmaps And Clustering - 15.7 Events

- 15.8 Routing And Distance - 15.9 A Complete Map Example

16. The Standard Library

- 16.1 Built-in Function Table - 16.2 I/O - 16.3 Math - 16.4 Collections - 16.5 Strings - 16.6 Time - 16.7 Network - 16.8 System - 16.9 Encoding - 16.10 Cryptography

17. Tooling

- 17.1 The Web IDE - 17.2 VS Code Extension - 17.3 CLI Reference - 17.4 Bytecode Compiler - 17.5 The sdev Assistant - 17.6 Linter - 17.7 Formatter - 17.8 Debugger - 17.9 Test Runner

18. Advanced Topics

- 18.1 JavaScript Interop - 18.2 Matrix Math - 18.3 Buffers and Pointers - 18.4 The Virtual Kernel - 18.5 Garbage Collection - 18.6 The Bytecode VM - 18.7 Multi-Language Source - 18.8 Embedding sdev - 18.9 FFI

19. Performance

- 19.1 Choose The Right Runtime - 19.2 Profile Before Optimising - 19.3 Avoid Quadratic Patterns - 19.4 Cache Computations - 19.5 Reuse Buffers - 19.6 Compile To Bytecode

20. Best Practices

- 20.1 Name Things Well - 20.2 Prefer Verbs - 20.3 Keep Functions Small - 20.4 Embrace Immutability - 20.5 Document Intent - 20.6 Test Early, Test Often - 20.7 Fail

Loudly - 20.8 Prefer Composition Over Inheritance

21. A Cookbook of Examples

- 21.1 FizzBuzz - 21.2 Word Count - 21.3 Frequency Histogram - 21.4 Todo App UI - 21.5 HTTP Fetch - 21.6 Bouncing Ball Animation - 21.7 Palindrome Checker - 21.8 Fibonacci (Memoised) - 21.9 Calculator UI - 21.10 Web Scraper - 21.11 Map Of Capitals

22. Building A Real Application

- 22.1 Project Layout - 22.2 Storage Module - 22.3 Render Module - 22.4 UI - 22.5 Wiring It Up - 22.6 Adding Cloud Sync - 22.7 Publishing

23. Frequently Asked Questions

- 23.1 Is sdev fast? - 23.2 Can I use it in production? - 23.3 Why not just use Python? - 23.4 Why two runtimes? - 23.5 Where do I report bugs? - 23.6 Can I contribute? - 23.7 How do I keep my files between sessions? - 23.8 Why "forge" instead of "let"?

24. Glossary

- 24.1 A–F - 24.2 G–N - 24.3 O–Z

25. Design Notes

- 25.1 Verbs Over Symbols - 25.2 Why CODE0 And Not Indentation - 25.3 Two Runtimes, One Language - 25.4 Why A UI Toolkit Is Built In - 25.5 Multi-Language Source - 25.6 Inspirations

26. Appendices

- 26.1 A. Keyword Reference - 26.2 B. Operator Reference - 26.3 C. Built-in Function Index - 26.4 D.

Source Languages - 26.5 E. Bytecode Opcodes - 26.6 F. Syscall Table - 26.7 G. Error Code Index - 26.8 H. CLI Flag Reference - 26.9 I. Index of Examples

1. Introduction

Welcome to **The sdev Book** — the definitive, comprehensive reference for the sdev programming language. sdev is a modern, expressive language with a uniquely human-readable syntax (`forge`, `be`, `speak`, `conjure`, `summon`) designed to make code feel like prose without sacrificing power, performance, or expressiveness.

This book is *long*. That is intentional. Programming languages are rich worlds, and sdev — with its readable syntax, dual runtimes, UI toolkit, kernel, bytecode VM, and 26-language source support — has more surface area than most. We want one document you can keep open and trust.

1.1 Why sdev?

sdev was created to bridge the gap between *talking about code* and *writing code*. Traditional languages force you into algebraic shorthand (`x = 1`, `def`, `var`). sdev replaces these with verbs and nouns that carry meaning: `forge x be 1`. The result is code that reads like a recipe, an essay, or a spell — depending on your style.

sdev also takes seriously the idea that **a language should ship with everything you need**: a UI toolkit, a graphics canvas, a mapping API, a kernel-style task scheduler, a bytecode VM, AI-aware tooling, and translations of the keywords into 26 spoken languages. You should not need to install five libraries before drawing a circle.

1.2 A Brief History

sdev started as an experiment in *prose-shaped* programming — could a language with `forge x be 1` feel as natural to read aloud as English? The first interpreter was a 400-line tree-walker. From there it grew into two parallel runtimes (JavaScript and Python), a bytecode compiler, a virtual kernel, and an in-browser IDE. Today sdev is used for teaching, scripting, prototyping interactive UIs, and producing maps and data visualisations.

1.3 Who Should Read This Book

This book is for everyone:

- **Complete beginners** learning to program for the first time. Chapters 1–6 are written gently, with annotated examples.
- **Professional developers** exploring an expressive new language. Skim the basics and dive into Chapters 11+ for concurrency, the kernel, and

bytecode internals.

- **Educators** teaching computational thinking. The verb-based vocabulary maps cleanly onto English (and 25 other languages).
- **Language designers** studying alternative syntactic paradigms. Chapter 27 dissects design decisions.

Each chapter starts gentle and ramps up.

1.4 How To Use This Book

Read sequentially the first time. Skim later. Every example is runnable in the **online IDE** at https://s-dev.lovable.app or with the offline Python/JS interpreters available from the Downloads menu.

The single best way to learn sdev is to **type the examples** — do not just read them. Modify them. Break them on purpose. When something does not behave as expected, that is when you learn.

1.5 Conventions

Code blocks use the `sdev` syntax. Output is shown in `// =>` comments. Notes labeled **TIP**, **WARNING**, **DEEP DIVE**, and **HISTORY** signal important context. Bulgarian translations of every chapter are in `sdev-book-bg.md`.

We write `forge x be 1` rather than `forge x = 1` throughout. Both are valid; `be` is preferred because it

reads aloud naturally.

1.6 A First Taste

```
// A complete sdev program.
forge greet be conjure(name): "Hello, " + name end
speak(greet("world"))
// =&gt; Hello, world
```

If that looks unusual, good. Read on; in five chapters it will feel obvious.

2. Getting Started

In this chapter you will install sdev (or skip installation by using the web IDE), write your first program, and understand the moving parts of a project.

2.1 Installing sdev

There are several ways to run sdev:

1. **Web IDE** — no install, open https://s-dev.lovable.app/ide. Works on any modern browser.
2. **JavaScript runtime** — `node sdev-interpreter.js my-file.sdev`. A single ~120 KB file with no dependencies.
3. **Python runtime** — `python sdev-interpreter.py my-file.sdev`. Useful when embedding into Python

tooling.

4. **VS Code extension** — `.vsix` from the Downloads menu. Adds syntax highlighting, snippets, and a Run command.
5. **Windows installer** — bundled batch installer with a tiny editor.

All runtimes share strict feature parity. A program that runs in one will run in the other, with the exception of host-only APIs (Leaflet maps require the JS runtime + a browser DOM).

2.2 Your First Program

Create a file `hello.sdev`:

```
speak("Hello, world!")
```

Run it:

```
node sdev-interpreter.js hello.sdev
# =&gt; Hello, world!
```

Congratulations — you are an sdev programmer.

2.3 The REPL

Start the REPL with `python sdev-interpreter.py` (no arguments). Type expressions and they will be evaluated immediately:

```
sdev&gt; forge x be 10
```

```
sdev&gt; x * 2
20
sdev&gt; :quit
```

Useful REPL meta-commands: `:quit`, `:reset`, `:load file.sdev`, `:save session.sdev`, `:type expr`.

2.4 Project Structure

A typical sdev project has:

- `main.sdev` — entry point
- `lib/` — your modules
- `assets/` — images, data files
- `README.md` — documentation
- `manifest.sdev` — optional package manifest for `summon`

There is no required layout. sdev is happy with a single file.

2.5 Running From the IDE

In the web IDE, press `Ctrl+Enter` (or `Cmd+Enter` on macOS) to run the active tab. The output panel shows printed text; the **App** panel shows UI windows; the **Canvas** panel shows graphics; the **Map** panel shows Leaflet output. The IDE auto-saves to your account if you are signed in.

2.6 Anatomy of a Program

An sdev source file is a sequence of statements. Statements include declarations, expressions, control structures, function and class definitions, and imports. Whitespace is significant only inside strings; indentation is for humans, `end` is for the parser.

2.7 Editor Setup

For non-IDE workflows, the VS Code extension bundles syntax + snippets. Vim users can drop the `sdev.vim` file into `~/.vim/syntax/`. Emacs users can use `sdev-mode.el` from the extension folder.

2.8 Common Pitfalls For Beginners

- Forgetting `end` after `if`, `while`, or `conjure`. The error message will tell you the line of the unmatched opener.
- Using `=` instead of `be` for assignment. Both work; `be` is canonical.
- Mixing `yes`/`no` with `true`/`false`. Both are accepted; pick one and stay consistent in a file.

3. Variables and Values

Every program manipulates data. In sdev you create variables with `forge` and bind them with `be`. Values are typed dynamically — sdev figures out the type at

runtime, but you can inspect, convert, and constrain types as needed.

3.1 Forging Variables

```
forge name be "Ada"
forge age be 36
forge alive be yes
```

The `forge` keyword introduces a *new* binding in the current scope. Re-assignment uses just the name:

```
name be "Lovelace"
```

Reassignment without `forge` keeps the variable in its original scope. This rule prevents accidental shadowing of outer-scope variables.

3.2 The Six Primitive Types

sdev recognises six primitive types:

Type	Examples	Notes		---	---	---
number	`42`, `3.14`, `-7`	One numeric type covers ints and floats.
string	`"hi"`, `'hi'`	Unicode by default.
boolean	`yes`, `no` (or `true`/`false`)	
void	`nothing`	The absence of a value.
collection	`[1,2]`, `{a: 1}`	Lists and dictionaries.
function	`conjure(): ... end`	First-class.

3.3 Type Inspection

Use `essence(value)` to ask what type a value is.

```
speak(essence(42))      // =&gt; "number"
speak(essence("hi"))    // =&gt; "string"
speak(essence(yes))     // =&gt; "boolean"
speak(essence([1,2]))   // =&gt; "collection"
speak(essence(speak))   // =&gt; "function"
```

3.4 Type Conversion (`morph`)

`morph(value, type_name)` converts between types when sensible.

```
forge n be morph("42", "number")
speak(n + 1)  // =&gt; 43
forge s be morph(3.14, "string")
speak(s + " is pi")  // =&gt; "3.14 is pi"
```

If the conversion is impossible, `morph` raises a runtime error. Wrap with `attempt/rescue` to handle gracefully.

3.5 Constants (`eternal`)

Prefix with `eternal` to mark a binding immutable.

```
eternal forge PI be 3.14159
PI be 4   // =&gt; error: cannot rebind eternal binding
```

TIP: use `eternal` liberally. Future readers (including your future self) benefit from knowing what does not change.

3.6 Naming Rules

Identifiers must begin with a letter or underscore and may contain letters, digits, and underscores. Reserved keywords cannot be used. snake_case is conventional for variables and functions; PascalCase for classes; SCREAMING_CASE for `eternal` constants.

3.7 Scope

sdev uses **lexical scoping**. A `forge` inside a block creates a new local binding that shadows outer bindings. The outer binding is restored when the block ends. Functions capture variables by reference, creating closures.

3.8 Nothing, Truthiness, Equality

`nothing` is the only value of type `void`. In boolean contexts, the falsy values are: `no`, `nothing`, `0`, `""`, and the empty collection `[]`/`{}`. Everything else is truthy.

`equals` (alias `==`) compares by value for primitives and by structure for collections. `same` compares by reference identity.

3.9 Numbers In Detail

Numbers are 64-bit floating point internally, with a transparent integer fast-path. Integer arithmetic

preserves precision up to 2^53. Use `bigint(n)` to opt into arbitrary-precision integers.

3.10 Strings In Detail

Strings are immutable Unicode. Indexing returns a single-character string. Negative indices count from the end. Common operations: `upper`, `lower`, `trim`, `reverse`, `contains`, `replace`, `shatter`, `weave`. Triple-quoted strings preserve newlines.

4. Operators and Expressions

sdev supports the usual arithmetic, comparison, and logical operators, with English-word aliases for prose-style code.

4.1 Arithmetic

Standard symbols: `+`, `-`, `*`, `/`, `%` (modulo), `**` (exponent). Integer division uses `//`. Unary `-` negates.

4.2 Comparison

`==`, `!=`, `<`, `>`, `<=`, `>=`. The aliases `equals`, `differs`, `below`, `above`, `belowOr`, `aboveOr` are available for prose-style code:

```
if age above 18 then speak("Adult") end
```

4.3 Logical

`and`, `or`, `not`. Short-circuit evaluation is guaranteed. The symbols `&&`, `||`, `!` also work but are less idiomatic.

4.4 String Operators

Concatenate with `+`. Repeat with `*`:

```
speak("ha" * 3)  // =&gt; hahaha
```

Interpolation uses backticks:

```
forge n be 7
speak(`There are ${n} cats`)
```

4.5 Membership

`x in coll` returns `yes` if `x` is in a collection, key in a dictionary, or substring in a string.

4.6 Pipelines (`|>`)

The pipeline operator passes a value as the first argument of the next function:

```
"  Hello  " |&gt; trim |&gt; upper |&gt; speak  // =&gt; HELLO
```

4.7 Operator Precedence

From tightest to loosest: `**`, unary `-`/`not`, `* / // %`, `+ -`, comparisons, `and`, `or`, `|>`, assignment. Parentheses always win.

4.8 Truthy Conversions

Boolean operators return *the value*, not a boolean: `forge x be value or "default"` is a common idiom for fallbacks.

5. Control Flow

sdev offers familiar control structures with an English-friendly twist.

5.1 If / Then / Else

```
if score above 90 then
  speak("Excellent")
else if score above 70 then
  speak("Good")
else
  speak("Try again")
end
```

5.2 When (Pattern Match)

A multi-branch conditional that reads top-to-bottom:

```
when:
  is x below 0 then speak("negative")
```

```
  is x equals 0 then speak("zero")
  is x above 0 then speak("positive")
  else speak("not a number")
end
```

5.3 While And Until

```
while count below 10:
  speak(count)
  count be count + 1
end

until ready:
  poll()
end
```

5.4 For ... In

```
for name in ["Ada", "Grace", "Linus"]:
  speak("Hi " + name)
end

for i in sequence(1, 10):
  speak(i)
end
```

5.5 Escape And Skip

`escape` exits the innermost loop; `skip` jumps to the next iteration.

```
for n in sequence(1, 100):
  if n % 2 == 0 then skip end
  if n above 20 then escape end
```

```
  speak(n)
end
```

5.6 Labelled Loops

Prefix a loop with `label name:` and use `escape name` or `skip name` to break out of nested loops directly.

5.7 Match (Destructuring)

Extract from collections inline:

```
forge [first, *rest] be [1, 2, 3, 4]
forge {name, age} be {name: "Ada", age: 36}
```

6. Functions

Functions are first-class values in sdev. Declare with `conjure`; pass them around like any other data.

6.1 Defining a Function

```
conjure greet(name be "stranger"):
  speak("Hello, " + name)
end

greet()        // =&gt; Hello, stranger
greet("Ada")   // =&gt; Hello, Ada
```

6.2 Default and Keyword Arguments

Defaults attach with `be`. Callers may use positional or keyword form:

```
conjure rect(width be 10, height be 5):
  return width * height
end
speak(rect(height: 8))  // =&gt; 80
```

6.3 Variadic Arguments

`*items` collects extra positional arguments. `**opts` collects keyword arguments.

6.4 Returning Values

Use `return value` to exit early. The last expression in a function is its implicit return value.

6.5 Anonymous Functions (Lambdas)

```
forge double be (n) =&gt; n * 2
speak(double(7))  // =&gt; 14
```

Lambdas capture surrounding variables, forming closures.

6.6 Closures

```
conjure counter():
```

```
  forge n be 0
  return () =&gt; { n be n + 1; return n }
end
forge tick be counter()
speak(tick())  // =&gt; 1
speak(tick())  // =&gt; 2
```

6.7 Higher-Order Functions

`each`, `sift`, `fold`, `seek`, `every`, `some` accept functions. They form the backbone of idiomatic data processing.

6.8 Recursion

sdev supports unbounded recursion (subject to the host stack). Use the `tailcall` modifier to enable tail-call optimisation:

```
conjure tailcall sum_to(n, acc be 0):
  if n equals 0 then return acc end
  return sum_to(n - 1, acc + n)
end
```

6.9 Generators (`yield`)

Functions that `yield` produce iterators on demand:

```
conjure naturals():
  forge n be 0
  while yes:
    yield n
    n be n + 1
  end
```

```
end
```

6.10 Decorators

A decorator wraps a function with extra behaviour. Apply with `@`:

```
@logged
conjure work(): speak("doing work") end
```

7. Collections

Collections are how sdev programs store many values together. The two core kinds are **lists** and **dictionaries**.

7.1 Lists

```
forge nums be [1, 2, 3]
speak(nums[0])              // =&gt; 1
speak(nums[-1])             // =&gt; 3
gather(nums, 4)             // [1,2,3,4]
forge head be pluck(nums, 0)
```

7.2 Dictionaries

```
forge user be {name: "Ada", age: 36}
speak(user.name)           // =&gt; Ada
speak(user["age"])         // =&gt; 36
user.email be "ada@x.com"
```

7.3 Slicing

`portion(coll, start, end?)` returns a sub-collection. Negative indices count from the end. Omit `end` to slice to the end.

7.4 Map / Filter / Reduce

```
forge squares be each([1,2,3,4], (n) =&gt; n * n)
forge evens be sift([1,2,3,4], (n) =&gt; n % 2 equals 0)
forge total be fold([1,2,3,4], 0, (a, b) =&gt; a + b)
```

7.5 Sets

Use `unique(coll)` to deduplicate. For frequent membership tests, `set(coll)` returns a hash-set wrapper.

7.6 Tuples

Immutable, fixed-length collections written `(a, b)`. Useful for returning multiple values:

```
conjure divmod(a, b): return (a // b, a % b) end
forge (q, r) be divmod(17, 5)
```

7.7 Iteration Idioms

Use `for ... in`, `enumerate`, and `zip` to traverse collections. Generators are lazy; lists are eager.

7.8 Performance Notes

Lists are dynamic arrays; appends are amortised O(1). Dictionaries are open-address hash maps; lookup is amortised O(1). Avoid `pluck(coll, 0)` in hot loops — it is O(n). Use a deque (`deque()`) for fast front operations.

8. Strings In Depth

Strings get a chapter of their own because text is everywhere — input, output, file paths, network protocols, UI labels.

8.1 Literals

Single quotes (`'hi'`) and double quotes (`"hi"`) are equivalent. Triple quotes (`"""..."""`) preserve newlines. Backticks support `${interpolation}`.

8.2 Indexing And Slicing

Strings are indexable like lists. They are immutable; operations return new strings.

8.3 Searching

Use `contains(s, sub)`, `indexOf(s, sub)`, `lastIndexOf(s, sub)`. For regex, use `regex(pattern, s)`.

8.4 Manipulation

`upper`, `lower`, `trim`, `lstrip`, `rstrip`, `replace(s, find, with)`, `shatter(s, sep)`, `weave(coll, sep)`.

8.5 Encoding

`base64encode/decode`, `urlencode/decode`, `hexencode/decode`. All accept strings or byte buffers.

8.6 Formatting

Use backtick interpolation, or `format(template, args)` with `%s`, `%d`, `%.2f` style placeholders.

8.7 Regular Expressions

```
forge m be regex("(\\w+)@(\\w+)", "ada@example")
speak(m.groups[0])  // =&gt; ada
```

8.8 Unicode And Locales

sdev strings are sequences of Unicode codepoints. `lower` and `upper` are locale-aware when a locale is set with `locale("bg_BG")`.

9. Object-Oriented Programming

sdev includes a full OOP system built on the keywords `essence` (define a class), `summon` (instantiate), `share` (declare instance/class fields), `extends` (inherit), `me` (this), and `ancestor` (super).

9.1 Defining a Class

```
essence Dog:
  share name
  share breed be "mutt"

  conjure init(name, breed be "mutt"):
    me.name be name
    me.breed be breed
  end

  conjure bark():
    speak(me.name + " woofs!")
  end
end
```

9.2 Creating Instances

```
forge rex be summon Dog(name: "Rex", breed: "Labrador")
rex.bark()
```

9.3 Inheritance

```
essence Puppy extends Dog:
  conjure bark():
    ancestor.bark()
    speak("(tiny)")
  end
end
```

9.4 Class Methods And Fields

Prefix with `static` to attach to the class rather than instances:

```
essence Counter:
  static share total be 0
  static conjure tick(): Counter.total be Counter.total + 1 end
end
```

9.5 Mixins (`weave`)

Include reusable methods from another module:

```
essence Bird weave Flyable, Singable: end
```

9.6 Operator Overloading

Implement special methods like `__add__`, `__eq__`, `__str__` to customise built-in operators on your class.

9.7 Properties

Decorate a method with `@property` to expose it as a field. `@setter` makes it writable.

9.8 Abstract Classes And Interfaces

Mark a method `abstract conjure name()` to require subclasses to provide it. `interface` declares only signatures.

10. Error Handling

Programs fail. sdev gives you precise, ergonomic tools to handle failure.

10.1 Raising Errors

```
raise "Out of fuel"
raise FuelError("Out of fuel", code: 42)
```

10.2 Attempt / Rescue / Finally

```
attempt:
  forge data be readFile("missing.txt")
rescue err:
  speak("Could not read: " + err.message)
finally:
  speak("done")
end
```

10.3 Error Types

Built-in error types include `TypeError`, `ValueError`, `IOError`, `KeyError`, `IndexError`, `NetworkError`, `TimeoutError`, `SyntaxError`. Define your own by extending `essence Error`.

10.4 Multiple Rescues

A single `attempt` may have several `rescue` clauses, each matched by error type:

```
attempt:
  doWork()
rescue NetworkError as e:
  retry()
rescue err:
  raise err
end
```

10.5 Re-raising

Inside a `rescue`, call `raise` with no argument to re-throw the current error. Useful for logging then propagating.

10.6 Stack Traces

Errors carry a `.trace` field — a list of frames with file, line, and function name. `speak(err.trace)` prints a formatted trace.

10.7 Assertions

Use `ensure(condition, "message")` to validate invariants. Failed assertions raise `AssertionError`.

10.8 Defensive Programming Idioms

Validate inputs at function boundaries with `ensure`. Use `attempt/rescue` only at boundaries you genuinely intend to recover at — not as silent error swallowers.

11. Concurrency

sdev embraces cooperative concurrency. Tasks share a single thread but yield to one another at well-defined points.

11.1 Async Functions

Any function may be `async`. Inside, use `await` to wait for an asynchronous result without blocking other tasks.

11.2 Dispatch And Await

```
async conjure fetchUser(id):
  forge res be await fetch("/users/" + id)
  return jsondecode(res.body)
end

forge promise be dispatch fetchUser(7)
forge user be await promise
speak(user.name)
```

11.3 Parallel Composition

`await all([p1, p2])` waits for several promises in parallel. `await any([p1, p2])` returns the first to resolve. `await race([p1, p2])` returns the first settled (resolve or reject).

11.4 Channels

`channel()` creates a queue. `send(ch, v)` and `forge v be await receive(ch)` move values between tasks.

11.5 Cancellation

Promises support `cancel(p)`. Inside `attempt`, `rescue CancelledError` cleans up gracefully.

11.6 Time And Delay

`await delay(1000)` pauses for one second. `every(ms, fn)` schedules a repeating task.

11.7 The Kernel

Under the hood, all tasks live in a virtual kernel with its own scheduler. See Chapter 18 for syscalls and direct task spawning.

12. Modules and Libraries

Real programs span multiple files. sdev modules are simply other `.sdev` files. The `summon` keyword imports them, locally or from the network.

12.1 Local Modules

`summon "./utils.sdev" as u` makes `u.fnName` available. Symbols can be selectively imported: `summon "./utils.sdev" {add, mul}`.

12.2 Gist Packages

sdev uses **GitHub Gists** as a decentralised package registry. `summon "gist:abc123/util.sdev" as util` fetches and caches a gist on first use.

12.3 Versioning

Pin a gist to a specific revision: `summon "gist:abc123@v1.2.0/util.sdev"`. Without a version, the latest revision is used and cached for 24 hours.

12.4 Manifests

A `manifest.sdev` file may declare dependencies. Run `sdev install` to fetch them all upfront.

12.5 Authoring A Gist Module

Any file with public `conjure` and `essence` declarations is a valid module. Use `share private` to hide internals.

12.6 Module Scope

Each module has its own top-level scope. `summon` does not pollute the importer's globals unless you write `summon * from ...`.

12.7 Dynamic Loading

Use `loadModule(url)` to import at runtime. Useful for plugins.

13. The UI Toolkit

sdev ships with a complete declarative UI toolkit for building windows, forms, dashboards, and small apps. The same code runs in the IDE App panel and (via the JS interpreter + DOM) in the browser.

13.1 Windows

`window(title, width, height)` opens a window. End with `endwindow`. Width and height are CSS pixels.

```
window("My App", 480, 600)
```

```
  heading("Welcome", 1)
  paragraph("Hello there.")
endwindow
```

13.2 Headings, Labels, Paragraphs

`heading(text, level=1)` for titles (levels 1–6). `label(text)` for short lines. `paragraph(text)` for body copy.

13.3 Buttons

```
button("Click me", () =&gt; speak("clicked"), "primary")
```

Variants: `primary`, `secondary`, `ghost`, `destructive`, `link`. Pass an `icon:` keyword for an icon prefix.

13.4 Inputs

```
input("name", "Your name")
input("age", "Age", "number")
input("password", "Password", "password")
```

The first argument is the *bind name* — read it later with `uiget("name")`.

13.5 Textareas

`textarea(bind, placeholder, rows)` for multi-line input.

13.6 Checkboxes And Radios

```
checkbox("agree", "I agree")
radio("plan", "free", "Free")
radio("plan", "pro", "Pro")
```

13.7 Sliders And Selects

```
slider("volume", 0, 100, 1)
select("country", ["BG", "US", "DE"], "Pick one")
```

13.8 Tables

```
table(["Name", "Age"], [
  ["Ada", 36],
  ["Grace", 85],
])
```

13.9 Tabs

```
tabs()
  tab("Home")
    paragraph("Welcome.")
  endtab
  tab("Settings")
    checkbox("dark", "Dark mode")
  endtab
endtabs
```

13.10 Menus

```
menu("File")
  menuitem("New", () =&gt; newDoc())
  menuitem("Open", () =&gt; openDoc())
endmenu
```

13.11 Layout: Rows, Columns, Groups

Use `row()/endrow` for horizontal stacks, `column()/endcolumn` for vertical, `group("title")/endgroup` for bordered cards.

13.12 Reactive Values

All bound widgets read from and write to a reactive store. `uiget(name)` reads; `uiset(name, value)` writes and re-renders the window.

13.13 Event Handlers

Buttons, menu items, and many widgets accept a callback. The callback runs in the same scope as the program; closures over outer variables work as expected.

13.14 Customisation

Pass a final dictionary of options to most widgets:

```
button("Open URL", () =&gt; {}, "primary", { url: "https://sdev.co
image("/logo.png", 200, 100, "Logo")
```

13.15 A Complete Example

```
forge todos be []
```

```
window("Todos", 360, 480)
  heading("My Todos", 2)
  input("new", "What to do?")
  button("Add", () =&gt; {
    gather(todos, uiget("new"))
    uiset("new", "")
  }, "primary")
  divider()
  for t in todos:
    row()
      checkbox("done_" + t, t)
    endrow
  end
endwindow
```

14. Graphics and Canvas

For drawing and animation, sdev provides a 2D canvas API.

14.1 Opening A Canvas

`canvas(width, height)` opens a drawing surface in the IDE Canvas panel.

14.2 Drawing Primitives

`fill(color)`, `stroke(color)`, `lineWidth(n)`, `rect(x,y,w,h)`, `circle(x,y,r)`, `line(x1,y1,x2,y2)`, `polygon([pts])`, `text(s,x,y)`.

14.3 Colors

Accept named CSS colors, `#rrggbb`, `rgb(r,g,b)`, `hsl(h,s,l)`, or `rgba(r,g,b,a)`.

14.4 Animation

`frame(fn)` calls `fn(t)` once per frame with the elapsed time. `requestStop()` halts the loop.

14.5 Images And Sprites

`loadImage(url)` returns a future image. `drawImage(img, x, y, w, h)` draws it.

14.6 Transforms

`translate(dx,dy)`, `rotate(rad)`, `scale(sx,sy)`. Save and restore state with `push()` / `pop()`.

14.7 Mouse And Keyboard

`onmouse((evt) => ...)`, `onkey((evt) => ...)`. Events carry `x`, `y`, `key`, `button`.

14.8 Bouncing Ball Example

```
canvas(400, 400)
forge x be 200; forge y be 200; forge dx be 3; forge dy be 2
frame((t) =&gt; {
```

```
    fill("black"); rect(0, 0, 400, 400)
    fill("cyan"); circle(x, y, 20)
    x be x + dx; y be y + dy
    if x below 0 or x above 400 then dx be -dx end
    if y below 0 or y above 400 then dy be -dy end
  })
```

15. Maps and GIS

sdev integrates with Leaflet for interactive maps. The mapping API is host-specific (browser only).

15.1 Opening A Map

`map(lat, lon, zoom)` opens a map centred on the given coordinates.

15.2 Markers And Popups

`marker(lat, lon, "label")` drops a marker. `popup(lat, lon, "html")` shows a popup.

15.3 Tile Layers

`tiles(url_template, opts)` switches the basemap. Built-in styles: `osm`, `dark`, `satellite`.

15.4 Polylines And Polygons

`polyline([[lat,lon], ...], "color")`. `polygon([[lat,lon], ...], "color")`.

15.5 GeoJSON

`geojson(url_or_object)` adds an entire GeoJSON feature collection.

15.6 Heatmaps And Clustering

`heatmap(points)` and `cluster(points)` visualise dense data.

15.7 Events

`onclick((lat, lon) => ...)` reacts to map clicks. `onzoom((z) => ...)` reacts to zoom changes.

15.8 Routing And Distance

`distance((lat1,lon1), (lat2,lon2))` returns metres. `route(from, to)` returns a polyline via OSRM.

15.9 A Complete Map Example

```
map(42.6977, 23.3219, 12)
marker(42.6977, 23.3219, "Sofia")
polyline([[42.6, 23.3], [42.7, 23.4]], "cyan")
```

16. The Standard Library

A reference of the most-used built-in functions. The complete list is in `SDEV_DOCUMENTATION.md`.

16.1 Built-in Function Table

| Function | Description | |---|---| | `speak(value)` | Print a value followed by a newline. | | `whisper(value)` | Print a value with no newline. | | `shout(value)` | Print a value in uppercase, attention-grabbing form. | | `ask(prompt)` | Read a line from the user (CLI) or prompt dialog (web). | | `readFile(path)` | Read the entire contents of a text file as a string. | | `writeFile(path, text)` | Write a string to a file, replacing previous contents. | | `appendFile(path, text)` | Append a string to the end of a file. | | `exists(path)` | Returns yes if the file or folder exists. | | `delete(path)` | Removes a file or folder. | | `listdir(path)` | Returns a collection of names inside a folder. | | `essence(value)` | Returns the runtime type as a string. | | `morph(value, type)` | Convert a value to another type. | | `magnitude(value)` | Length of a string or collection; absolute value of a number. | | `least(...)` | Smallest of the given values or collection. | | `greatest(...)` | Largest of the given values or collection. | | `root(n, k=2)` | k-th root of n; default square root. | | `ground(n)` | Floor; round toward negative infinity. | | `elevate(n)` | Ceiling; round toward positive infinity. | | `around(n, digits=0)` | Round to a number of decimal digits. | | `random()` | Random float in [0, 1). | | `randint(a, b)` | Random integer in [a,

b]. | | `choose(coll)` | Random element from a collection. | | `sin(x) / cos(x) / tan(x)` | Trigonometry in radians. | | `asin/acos/atan/atan2` | Inverse trig. | | `log(x, base=e)` | Logarithm. | | `exp(x)` | Exponential e^x. | | `pi / tau / e` | Math constants. | | `gather(coll, value)` | Append a value to a collection in place. | | `pluck(coll, index)` | Remove and return an element at index. | | `portion(coll, start, end?)` | Slice a collection. | | `each(coll, fn)` | Apply fn to each element. | | `sift(coll, predicate)` | Filter a collection. | | `fold(coll, init, fn)` | Reduce a collection to a single value. | | `seek(coll, predicate)` | First element matching predicate. | | `every(coll, predicate)` | Yes if all elements match. | | `some(coll, predicate)` | Yes if any element matches. | | `zip(a, b, ...)` | Pairs elements from multiple collections. | | `enumerate(coll)` | Pairs of (index, element). | | `sequence(start, end, step=1)` | A range of numbers. | | `sort(coll, key?)` | Sort a collection ascending. | | `reverse(coll)` | Reverse a collection or string. | | `unique(coll)` | Distinct values, preserving order. | | `shatter(s, sep)` | Split a string by separator. | | `weave(coll, sep)` | Join a collection with a separator. | | `upper(s) / lower(s) / trim(s)` | String case and whitespace. | | `contains(haystack, needle)` | Substring/element membership. | | `replace(s, find, with)` | Replace all occurrences. | | `regex(pattern, s)` | Regular expression match. | | `now()` | Current epoch milliseconds. | | `today()` | Current local date as ISO string. | | `delay(ms)` | Pause asynchronously for ms milliseconds. | | `format(date, pattern)` | Format a date. | | `env(name) / argv() / exit(code)` | Process and OS

interaction. | | `osinfo()` | Returns a dictionary about the host OS. | | `base64encode/decode, hexencode/decode` | Binary encodings. | | `jsonencode(v) / jsondecode(s)` | JSON serialization. | | `fetch(url, opts?)` | Async HTTP request. | | `matrix(rows) / matmul / inverse / transpose / det` | Linear algebra. | | `buffer(size) / pointer(v) / deref(p)` | Low-level memory primitives. | | `spawn(fn) / schedule() / syscall(name, args) / gc()` | Kernel / scheduler primitives. | | `window(title, w, h)` | Open a window. End with endwindow. | | `heading(text, level=1)` | A bold heading. | | `label(text)` | A line of text. | | `paragraph(text)` | A paragraph of body text. | | `button(label, onclick, variant?)` | Clickable button. Variants: primary, secondary, ghost, destructive. | | `input(bind, placeholder?, kind?)` | Text input bound by name. Kind: text, number, password, email, url. | | `textarea(bind, placeholder?, rows?)` | Multi-line input. | | `checkbox(bind, label)` | Boolean toggle with label. | | `radio(bind, value, label)` | Radio button bound by name. | | `slider(bind, min, max, step=1)` | Numeric slider. | | `select(bind, options, placeholder?)` | Dropdown of options. | | `image(src, w?, h?, alt?)` | Display an image. | | `link(text, url, newtab=yes)` | Open a URL. | | `progress(value, max=100)` | Determinate progress bar. | | `divider() / spacer(size?)` | Visual layout helpers. | | `row()/endrow, column()/endcolumn, group(title)/endgroup` | Layout containers. | | `tabs()/endtabs, tab(title)/endtab` | Tabbed sections. | | `table(headers, rows)` | A data table. | | `menu(title)/endmenu, menuitem(title, onclick)` | Menus

and items. | | `uiget(name) / uiset(name, value)` | Read or write a reactive value. |

16.2 I/O

`speak`, `whisper`, `shout`, `ask`, `readFile`, `writeFile`, `appendFile`, `exists`, `delete`, `listdir`.

16.3 Math

`magnitude`, `least`, `greatest`, `root`, `ground`, `elevate`, `around`, `random`, `randint`, `choose`, `sin/cos/tan`, `log`, `exp`, `pi`, `tau`, `e`.

16.4 Collections

`gather`, `pluck`, `portion`, `each`, `sift`, `fold`, `seek`, `every`, `some`, `zip`, `enumerate`, `sequence`, `sort`, `reverse`, `unique`, `set`, `deque`.

16.5 Strings

`upper`, `lower`, `trim`, `reverse`, `contains`, `replace`, `shatter`, `weave`, `regex`, `format`.

16.6 Time

`now`, `today`, `delay`, `every`, `format`, `parseDate`, `timezone`.

16.7 Network

`fetch`, `websocket`, `download`.

16.8 System

`env`, `argv`, `exit`, `osinfo`, `cwd`, `cd`.

16.9 Encoding

`base64encode/decode`, `hexencode/decode`, `urlencode/decode`, `jsonencode/decode`, `csv`, `xml`.

16.10 Cryptography

`hash(s, alg="sha256")`, `hmac(key, msg, alg)`, `uuid()`, `randomBytes(n)`.

17. Tooling

Editors, linters, debuggers, and the bytecode compiler.

17.1 The Web IDE

A full-featured browser IDE at https://s-dev.lovable.app/ide. Includes a file tree, tabs, command palette, terminal, output, App preview, canvas, map, settings, and a chatbot.

17.2 VS Code Extension

Download `.vsix` from the Downloads menu. Provides syntax highlighting, snippets, and a Run command (`F5`).

17.3 CLI Reference

`node sdev-interpreter.js [opts] [file]`

- `--repl` start REPL
- `--compile file.sdev` produce `file.sdevc`
- `--exec file.sdevc` run bytecode
- `--lang bg` set source language
- `--ast` print parse tree
- `--time` print execution timing

17.4 Bytecode Compiler

Compile to `.sdevc` for faster startup: `node sdev-interpreter.js --compile main.sdev`. Run with `--exec main.sdevc`.

17.5 The sdev Assistant

In-IDE AI chatbot that explains code, generates examples, and debugs runtime errors via a self-test loop.

17.6 Linter

`sdev lint file.sdev` checks for unused variables, missing `end`s, suspicious comparisons, and shadowed `eternal` bindings.

17.7 Formatter

`sdev format file.sdev` rewrites a file in canonical style. Idempotent.

17.8 Debugger

Place `breakpoint()` in code; in the IDE, execution pauses at that line and the variables panel becomes inspectable. Step with F10.

17.9 Test Runner

`sdev test` discovers `*_test.sdev` files and runs every `conjure test_*` function. Use `expect(value).toBe(other)` for assertions.

18. Advanced Topics

Power-user features for systems, performance, embedding, and unusual workflows.

18.1 JavaScript Interop

Embed raw JS with the `js` keyword:

```
js { console.log("native") }
```

Multi-line `js { ... }` blocks share scope with the surrounding sdev program. Values flow both ways.

18.2 Matrix Math

`matrix([[1,2],[3,4]])`, `matmul(a, b)`, `inverse(a)`, `transpose(a)`, `det(a)`. Useful for graphics, linear algebra, and ML toy projects.

18.3 Buffers and Pointers

`buffer(size)`, `pointer(value)`, `deref(p)`. For binary protocol work.

18.4 The Virtual Kernel

sdev includes a tiny virtual OS kernel. `spawn(fn)` creates a task; `schedule()` yields cooperatively; `syscall("name", args)` invokes a syscall (open, read, write, sleep, fork). The kernel offers a unified abstraction over the Node and browser hosts.

18.5 Garbage Collection

Mark-and-sweep GC runs automatically. Force a cycle with `gc()`. Inspect heap stats with `heapinfo()`.

18.6 The Bytecode VM

A stack-based VM executes `.sdevc` files. Opcodes include `LOAD_CONST`, `LOAD_NAME`, `STORE_NAME`, `CALL`, `JUMP`, `JUMP_IF_FALSE`, `RETURN`, `MAKE_FUNCTION`, `BUILD_LIST`, `BUILD_DICT`. The VM is ~3–5× faster than tree-walking.

18.7 Multi-Language Source

sdev source can be written in any of 26 languages. The translator normalises keywords at parse time. Set the source language with a `// lang: bg` directive at the top of the file, or `--lang bg` on the CLI.

18.8 Embedding sdev

Embed the JS interpreter in a web page with `import { run } from "sdev-interpreter.js"`. Embed the Python interpreter with `from sdev_interpreter import run`.

18.9 FFI

Call native libraries via `ffi.load("libm.so").func("sin", ["double"], "double")`. JavaScript host only.

19. Performance

sdev is fast enough for most scripts. When it is not, here is how to make it faster.

19.1 Choose The Right Runtime

Tree-walking interpreter: simplest, slowest. Bytecode VM: 3–5× faster. JS interop: native speed for hot loops.

19.2 Profile Before Optimising

Use `time` blocks: `time { work() }` prints elapsed milliseconds. Use `profile { work() }` to see a per-function breakdown.

19.3 Avoid Quadratic Patterns

Repeated `pluck(coll, 0)` or string concatenation in a loop is $O(n^2)$. Prefer `deque()` and `weave([...], "")` respectively.

19.4 Cache Computations

Memoise pure functions with `@memo`. Cache external fetches with `cache(ttl: 60)`.

19.5 Reuse Buffers

In tight loops, reuse a `buffer(N)` rather than allocating per iteration.

19.6 Compile To Bytecode

For long-running services, compile to `.sdevc`. Startup is faster and execution is faster.

20. Best Practices

Idioms and patterns that make sdev programs delightful to read and maintain.

20.1 Name Things Well

sdev rewards descriptive names. `forge n be 0` is fine for a counter inside a five-line loop; `forge customers_seen be 0` is better when shared across a function.

20.2 Prefer Verbs

Function names should be verbs (`compute`, `render`, `parse`); class names should be nouns (`Customer`, `Invoice`).

20.3 Keep Functions Small

Aim for under 20 lines. If a function does two things, split it.

20.4 Embrace Immutability

Use `eternal` liberally for values that should not change. Future readers will thank you.

20.5 Document Intent

Comments should explain *why*. The code already shows *what*.

20.6 Test Early, Test Often

Even one `expect(...)` per function pays for itself within a week.

20.7 Fail Loudly

Validate inputs at boundaries. Do not let bad data spread three layers deep before raising.

20.8 Prefer Composition Over Inheritance

Mixins (`weave`) and small classes outlast deep hierarchies.

21. A Cookbook of Examples

Short, complete programs that demonstrate idiomatic sdev. Type these in. Modify them. Break them.

21.1 FizzBuzz

```
for i in sequence(1, 100):
  when:
    is i % 15 == 0 then speak("FizzBuzz")
    is i % 3 == 0 then speak("Fizz")
    is i % 5 == 0 then speak("Buzz")
    else speak(i)
  end
end
```

21.2 Word Count

```
forge text be readFile("essay.txt")
forge words be shatter(text, " ")
speak("Word count: " + magnitude(words))
```

21.3 Frequency Histogram

```
forge counts be {}
for w in shatter(readFile("text.txt"), " "):
  counts[w] be (counts[w] or 0) + 1
end
for (k, v) in sort(enumerate(counts), key: (p) =&gt; -p[1]):
  speak(k + ": " + v)
end
```

21.4 Todo App UI

```
forge todos be []
window("Todos", 360, 480)
  heading("My Todos", 2)
```

```
  input("new", "What to do?")
  button("Add", () =&gt; {
    gather(todos, uiget("new"))
    uiset("new", "")
  }, "primary")
  divider()
  for t in todos: label("• " + t) end
endwindow
```

21.5 HTTP Fetch

```
async conjure main():
  forge res be await fetch("https://api.example.com/users")
  forge users be jsondecode(res.body)
  for u in users: speak(u.name) end
end
await main()
```

21.6 Bouncing Ball Animation

```
canvas(400, 400)
forge x be 200; forge y be 200; forge dx be 3; forge dy be 2
frame((t) =&gt; {
  fill("black"); rect(0, 0, 400, 400)
  fill("cyan"); circle(x, y, 20)
  x be x + dx; y be y + dy
  if x below 0 or x above 400 then dx be -dx end
  if y below 0 or y above 400 then dy be -dy end
})
```

21.7 Palindrome Checker

```
conjure palindrome(s):
  forge clean be lower(replace(s, " ", ""))
```

```
  return clean equals reverse(clean)
end
speak(palindrome("Race car"))  // =&gt; yes
```

21.8 Fibonacci (Memoised)

```
@memo
conjure fib(n):
  if n below 2 then return n end
  return fib(n - 1) + fib(n - 2)
end
speak(fib(50))
```

21.9 Calculator UI

```
window("Calc", 280, 320)
  input("a", "First number", "number")
  input("b", "Second number", "number")
  row()
    button("+", () =&gt; uiset("r", uiget("a") + uiget("b")))
    button("−", () =&gt; uiset("r", uiget("a") - uiget("b")))
    button("×", () =&gt; uiset("r", uiget("a") * uiget("b")))
    button("÷", () =&gt; uiset("r", uiget("a") / uiget("b")))
  endrow
  label("= " + (uiget("r") or "?"))
endwindow
```

21.10 Web Scraper

```
async conjure scrape(url):
  forge html be (await fetch(url)).body
  forge titles be regex("&lt;h2&gt;(.*?)&lt;/h2&gt;", html, all: yo
  return each(titles, (m) =&gt; m.groups[0])
end
```

21.11 Map Of Capitals

```
map(48.0, 10.0, 4)
for (city, lat, lon) in [("Sofia", 42.7, 23.3), ("Berlin", 52.5, 1
  marker(lat, lon, city)
end
```

22. Building A Real Application

A full-length walkthrough: from blank file to a working Markdown notebook with cloud sync.

22.1 Project Layout

We will create:

- `main.sdev` — entry
- `lib/storage.sdev` — local + cloud storage
- `lib/render.sdev` — Markdown renderer
- `ui/notebook.sdev` — UI window

22.2 Storage Module

```
// lib/storage.sdev
share conjure save(key, text):
  writeFile("notes/" + key + ".md", text)
end
share conjure load(key):
```

```
    return readFile("notes/" + key + ".md")
  end
```

22.3 Render Module

Use a tiny regex-based Markdown converter, or `summon "gist:abc/markdown.sdev"` for a full one.

22.4 UI

```
  window("Notebook", 720, 600)
    heading("Notebook", 2)
    input("title", "Note title")
    textarea("body", "Write your note...", 12)
    button("Save", () =&gt; storage.save(uiget("title"), uiget("body
  endwindow
```

22.5 Wiring It Up

`main.sdev` simply imports the modules and opens the window.

22.6 Adding Cloud Sync

When signed in, the workspace auto-syncs. Use `cloudSave("notes", todos)` for additional manual sync of arbitrary data.

22.7 Publishing

Export the project as a gist with `sdev publish` and share the URL.

23. Frequently Asked Questions

Real questions from real sdev users.

23.1 Is sdev fast?

For interpreted execution, performance is on par with Python. The bytecode compiler (`.sdevc`) gives a 3–5× speedup. For hot loops, drop into `js { ... }` interop.

23.2 Can I use it in production?

sdev is best suited for scripting, education, prototyping, internal tools, and embedded scenarios. Production web backends remain a stretch — but it is improving.

23.3 Why not just use Python?

Python is excellent. sdev exists for those who want a *more readable* alternative — closer to spoken English than algebraic shorthand — with a built-in UI toolkit, mapping, and 26-language source support.

23.4 Why two runtimes?

JavaScript runs everywhere a browser does — including the web IDE — and is great for UI/maps. Python is the default in scientific and pedagogic environments. Strict feature parity means you can pick either.

23.5 Where do I report bugs?

Open an issue on the GitHub repository linked from the website footer.

23.6 Can I contribute?

Yes! sdev is open source. Pull requests for new builtins, language features, and translations are welcome.

23.7 How do I keep my files between sessions?

Sign in. The IDE auto-syncs your entire workspace — files, folders, open tabs — to the cloud. Reload anywhere; pick up where you left off.

23.8 Why "forge" instead of "let"?

`forge` evokes creation. `let` evokes "permitting" — passive. Active verbs make code feel like *doing*, not *requesting*.

24. Glossary

Quick definitions of key terms.

24.1 A–F

Anonymous function — a function without a name, created with `=>`. **Bind** — to associate a name with a value. **Bytecode** — the compact instruction format produced by `--compile`. **Closure** — a function that captures variables from its lexical scope. **Conjure** — keyword to declare a function. **Eternal** — modifier for immutable bindings. **Forge** — keyword to declare a new variable.

24.2 G–N

Gist — a remote sdev module fetched via `summon`. **Kernel** — the virtual OS layer providing tasks and syscalls. **Lambda** — synonym for anonymous function. **Memo** — decorator that caches function results. **Module** — a `.sdev` file imported via `summon`. **Morph** — built-in for type conversion. **Mutation** — changing the value of a binding.

24.3 O–Z

Object — instance of a class. **Promise** — a value that will arrive later. **Reactive value** — a UI-bound value

managed by `uiget`/`uiset`. **Summon** — keyword to import a module or instantiate a class. **Tail call** — a function call in tail position; optimised when the function is `tailcall`. **VM** — the bytecode virtual machine. **Yield** — produce a value from a generator.

25. Design Notes

Why sdev looks the way it looks. Skip if you only want to write code.

25.1 Verbs Over Symbols

Symbols read fast for experts and slow for newcomers. Verbs read pleasantly aloud. We chose verbs.

25.2 Why `end` And Not Indentation

Significant indentation is fragile across editors and chats. Explicit `end` survives copy-paste, AI prompts, and email.

25.3 Two Runtimes, One Language

JavaScript and Python cover most environments where sdev would land. We accepted the cost of duplicating work to keep both groups happy.

25.4 Why A UI Toolkit Is Built In

A language without a way to draw a button is a language for tutorials. A language with one is a language for tools.

25.5 Multi-Language Source

Programming should not be gated on English literacy. The translator is bidirectional and lossless.

25.6 Inspirations

Lua, Python, Smalltalk, Ruby, and AppleScript. Anti-inspirations: Brainfuck, Perl golf, and templating DSLs.

26. Appendices

26.1 A. Keyword Reference

Complete keyword list:

`forge`, `be`, `eternal`, `if`, `then`, `else`, `when`, `is`, `while`, `until`, `for`, `in`, `escape`, `skip`, `conjure`, `yield`, `return`, `essence`, `summon`, `share`, `extends`, `me`, `ancestor`, `attempt`, `rescue`, `finally`, `raise`, `dispatch`, `await`, `js`, `end`, `window`, `endwindow`, `row`, `endrow`, `column`, `endcolumn`, `tabs`, `endtabs`, `tab`, `endtab`, `group`, `endgroup`, `menu`, `endmenu`, `menuitem`

26.2 B. Operator Reference

`+ - * / % ** //`, `== != < > <= >=`, `and or not`, `equals differs above below`, `in`, `=>`, `|>`.

26.3 C. Built-in Function Index

See Chapter 17 (The Standard Library). The complete index is in `SDEV_DOCUMENTATION.md`.

26.4 D. Source Languages

sdev source files may be written in: English, Bulgarian, Russian, Spanish, French, German, Italian, Portuguese, Polish, Romanian, Ukrainian, Greek, Turkish, Arabic, Hebrew, Hindi, Chinese (Simplified), Chinese (Traditional), Japanese, Korean, Vietnamese, Thai, Indonesian, Dutch, Swedish, Czech.

26.5 E. Bytecode Opcodes

`LOAD_CONST`, `LOAD_NAME`, `STORE_NAME`, `LOAD_ATTR`, `STORE_ATTR`, `LOAD_INDEX`, `STORE_INDEX`, `BINARY_OP`, `UNARY_OP`, `CALL`, `RETURN`, `JUMP`, `JUMP_IF_FALSE`, `JUMP_IF_TRUE`, `MAKE_FUNCTION`, `MAKE_CLOSURE`, `BUILD_LIST`, `BUILD_DICT`, `IMPORT`, `RAISE`, `TRY_BEGIN`, `TRY_END`, `YIELD`, `AWAIT`, `SPAWN`, `HALT`.

26.6 F. Syscall Table

`open`, `close`, `read`, `write`, `seek`, `stat`, `mkdir`, `rmdir`, `unlink`, `rename`, `sleep`, `fork`, `wait`, `kill`, `pipe`, `socket`, `connect`, `listen`, `accept`, `send`, `recv`, `time`, `getenv`, `setenv`, `gc`.

26.7 G. Error Code Index

`E_TYPE`, `E_VALUE`, `E_INDEX`, `E_KEY`, `E_IO`, `E_NET`, `E_TIMEOUT`, `E_SYNTAX`, `E_NAME`, `E_ATTR`, `E_DIVZERO`, `E_OVERFLOW`, `E_RECURSION`, `E_CANCELLED`, `E_INTERNAL`.

26.8 H. CLI Flag Reference

`--repl`, `--compile <file>`, `--exec <file>`, `--lang <code>`, `--ast`, `--time`, `--profile`, `--lint`, `--format`, `--test`, `--version`, `--help`.

26.9 I. Index of Examples

Bouncing ball (Ch.14), Calculator UI (Ch.20), Capitals map (Ch.20), FizzBuzz (Ch.20), Frequency histogram (Ch.20), Notebook app (Ch.21), Palindrome (Ch.20), Todo app (Ch.20), Web scraper (Ch.20), Word count (Ch.20).

End of book. Version 2.0. © sdev language.

CHAPTER 02

sdev Programming Language

sdev Programming Language

Complete Documentation, Tutorial & Reference Guide

Table of Contents

1. Introduction
2. Getting Started
3. Syntax Overview
4. Variables & Constants
5. Data Types
6. Operators
7. String Operations
8. Control Flow
9. Functions
10. Object-Oriented Programming
11. Error Handling
12. Async & Concurrency
13. Built-in Output Functions
14. Built-in Math Functions

15. Built-in String Functions
16. Built-in List Functions
17. Built-in Tome (Dict) Functions
18. Higher-Order Functions
19. Type System & Conversion
20. Collections: Set, Map, Queue, Stack, LinkedList
21. Matrix Operations
22. Graphics & Game Development
23. File I/O
24. Networking
25. Examples & Recipes
26. JavaScript Interop
27. Complete Reference Card

Introduction

sdev is an expressive, full-featured programming language with a deliberately unique syntax designed to feel fresh and readable. It supports:

- **Unique English-inspired keywords** — `forge`, `conjure`, `ponder`, `cycle`, `iterate`, `yield`, `yeet`, `skip`
- **Block delimiters** — `::` to open a block and `;;` to close it — no curly braces
- **First-class functions and lambdas** — `(x) -> x * 2`
- **Pipe operator** — `|>` for clean functional composition

- **Full OOP** — `essence` (classes), `extend` (inheritance), `self`, `super`, `new`
- **Error handling** — `attempt :: ... ;; rescue err :: ... ;;`
- **Built-in data structures** — Set, Map, Queue, Stack, LinkedList
- **Matrix math** — `matmul`, `transpose`, `dot`, `reshape`, etc.
- **2D Graphics & Turtle API** — canvas drawing, shapes, gradients, turtle graphics, sprites
- **Async / concurrency** — `async conjure`, `await`, `spawn`
- **JavaScript interop** — `js` keyword for seamless JS integration

sdev runs in the browser-based IDE (at `/ide`), in the Playground (`/`), and as a downloadable Electron desktop application.

Getting Started

Running in the IDE

Open the sdev IDE at `/ide`. Press **Ctrl+Enter** (or the **Run** button) to execute your program. Output appears in the **OUTPUT** panel at the bottom.

Running in the Playground

Go to the main Playground at `/`. Type or paste sdev code in the editor, then click **Run**.

Downloading the Desktop App

In the IDE, go to the **Download** button → **Electron Desktop App**. This gives you three files:

```
package.json
main.js
README.md
```

Install and run:

```
npm install
npm start
```

This opens sdev IDE as a native desktop window that wraps the full web IDE.

Your First Program

```
// hello.sdev
speak("Hello, World!")
```

Output:

```
Hello, World!
```

A slightly more involved first program:

```
forge name be "Alice"
forge age be 30

speak("Name:", name)
```

```
speak("Age:", age)
speak("In 10 years:", age + 10)

conjure greet(person, greeting) ::
  yield greeting + ", " + person + "!"
;;

speak(greet(name, "Welcome"))
```

Output:

```
Name: Alice
Age: 30
In 10 years: 40
Welcome, Alice!
```

Syntax Overview

Comments

```
// This is a single-line comment
# This is also a single-line comment (Python style)
```

Blocks

sdev uses `::` to start a block and `;;` to end it — never curly braces `{}`:

```
ponder x &gt; 5 ::
  speak("x is big")
;;

conjure add(a, b) ::
```

```
    yield a + b
  ;;
```

Blocks can also be written inline for short single-statement bodies:

```
ponder x &gt; 5 :: speak("big") ;;
```

No Semicolons Required

Statements are separated by newlines. You do NOT need `;` at the end of lines.

```
forge a be 1
forge b be 2
forge c be a + b
speak(c)
```

Indentation

Indentation is cosmetic but strongly recommended. The language uses `::` and `;;` for block structure, not whitespace.

Variables & Constants

Declaring Variables

Use `forge` to declare a new variable and `be` to assign its value:

```
forge name be "Alice"
forge age be 25
forge isActive be yep
forge score be 0.0
```

Reassigning Variables

Just use `be` without `forge`:

```
forge count be 0
count be count + 1
count be 10
```

Compound Assignment (shorthand patterns)

```
forge x be 10
x be x + 5    // x = 15
x be x * 2    // x = 30
x be x - 10   // x = 20
x be x / 4    // x = 5
```

Built-in Constants

sdev provides these constants directly:

```
speak(PI)        // 3.141592653589793
speak(TAU)       // 6.283185307179586 (2 * PI)
speak(E)         // 2.718281828459045
speak(INFINITY)  // Infinity
```

Multiple Variables

```
forge x be 10
forge y be 20
forge z be x + y

// Swap pattern
forge temp be x
x be y
y be temp
```

Data Types

sdev has six core types:

sdev Type	Description	Literal Example
`number`	Integer or float	`42`, `3.14`, `-7`, `1.5e10`
`text`	Strings	`"hello"`, `'world'`
`truth`	Booleans	`yep`, `nope`
`void`	Null / absent	`void`
`list`	Arrays	`[1, 2, 3]`
`tome`	Dictionaries	`:: "key": "value" ;;`

Functions and class instances are also first-class values.

Numbers

```
forge integer be 42
forge float be 3.14159
forge negative be -100
forge scientific be 1.5e10      // 15,000,000,000
forge hex be 0xFF               // 255
```

Arithmetic is standard, with `^` for power:

```
speak(2 ^ 10)   // 1024
```

```
speak(10 % 3)   // 1
speak(7 / 2)    // 3.5
```

Text (Strings)

```
forge single be 'Hello'
forge double be "World"

// Escape sequences
forge tab be "col1\tcol2"
forge newline be "line1\nline2"
forge quoted be "She said \"hi\""

// String concatenation
forge full be "Hello" + ", " + "World!"

// String repetition
forge stars be "*" * 10   // "**********"

// Access characters
speak("hello"[0])   // h
speak("hello"[-1])  // o (last character)
```

Truth (Booleans)

```
forge t be yep    // true
forge f be nope   // false

// Boolean expressions
speak(5 &gt; 3)          // yep
speak(5 equals 3)     // nope
speak(yep also nope)  // nope
speak(yep either nope) // yep
speak(isnt nope)       // yep
```

Void (Null)

```
forge nothing be void
ponder nothing equals void ::
  speak("It's void!")
;;
```

Lists

```
forge nums be [1, 2, 3, 4, 5]
forge mixed be [1, "two", yep, void]
forge empty be []
forge nested be [[1, 2], [3, 4], [5, 6]]

// Access by index (0-based)
speak(nums[0])    // 1
speak(nums[2])    // 3
speak(nums[-1])   // 5  (last element)

// Modify element
nums[0] be 100
speak(nums)       // [100, 2, 3, 4, 5]

// List length
speak(measure(nums))   // 5

// Slicing
speak(portion(nums, 1, 3))  // [2, 3]
speak(portion(nums, 2))     // [3, 4, 5]
```

Tomes (Dictionaries)

Tomes are key-value stores. Keys are strings.

```
forge person be ::
  "name": "Alice",
```

```
  "age": 30,
  "active": yep
;;

// Access values - both bracket and dot notation work
speak(person["name"])  // Alice
speak(person.age)      // 30

// Set new or existing keys
person["city"] be "New York"
person.age be 31

// Check if key exists
speak(contains(person, "name"))   // yep
speak(contains(person, "phone"))  // nope

// Get all keys, values, entries
speak(inscriptions(person))  // ["name", "age", "active", "city"]
speak(contents(person))      // ["Alice", 31, yep, "New York"]
speak(entries(person))       // [["name","Alice"], ...]
```

Operators

Arithmetic Operators

Operator	Description	Example	Result
`+`	Addition	`5 + 3`	`8`
`-`	Subtraction	`10 - 4`	`6`
`*`	Multiplication	`4 * 3`	`12`
`/`	Division	`7 / 2`	`3.5`
`%`	Modulo	`10 % 3`	`1`
`^`	Power	`2 ^ 8`	`256`
`-x`	Negation	`-5`	`-5`

Comparison Operators

Operator	Description	Example	Result
`equals`	Equal to	`5 equals 5`	`yep`
`differs`	Not equal	`5 differs 3`	`yep`
`<>`	Not equal (alt)	`5 <> 3`	`yep`
`<`	Less than	`3 < 5`	`yep`
`>`	Greater than	`5 > 3`	`yep`
`<=`	Less or equal	`3 <= 3`	`yep`
`>=`	Greater or equal	`5 >= 5`	`yep`

Logical Operators

Operator	Meaning	Example
`also`	AND	`yep also yep` → `yep`
`either`	OR	`nope either yep` → `yep`
`isnt`	NOT	`isnt nope` → `yep`

Short-circuit evaluation applies: `also` stops at first `nope`, `either` stops at first `yep`.

String & List Concatenation

```
"Hello" + " World"    // "Hello World"
[1, 2] + [3, 4]       // [1, 2, 3, 4]
"ha" * 3              // "hahaha"
[0] * 5               // [0, 0, 0, 0, 0]
```

Pipe Operator `|>`

The pipe operator passes the left-hand value as the first argument to the right-hand function:

```
// Without pipe
forge result be weave(sort(sift(each([1,2,3,4,5], x -&gt; x * 2), x

// With pipe — much cleaner!
```

```
forge result be [1, 2, 3, 4, 5]
  |&gt; each(x -&gt; x * 2)
  |&gt; sift(x -&gt; x &gt; 4)
  |&gt; weave(", ")
speak(result)  // "6, 8, 10"
```

Ternary Operator ~

Inline if/else expression:

```
forge x be 10
forge label be x &gt; 5 ~ "big" : "small"
speak(label)  // "big"

// In function calls
speak(magnitude(-5) &gt; 3 ~ "large magnitude" : "small magnitude"

// Nested ternary
forge grade be 85
forge letter be grade &gt;= 90 ~ "A" : grade &gt;= 80 ~ "B" : grad
speak(letter)  // "B"
```

Operator Precedence (high to low)

1. `^` (power)
2. Unary `-`, `isnt`
3. `*`, `/`, `%`
4. `+`, `-`
5. `<`, `>`, `<=`, `>=`
6. `equals`, `differs`, `<>`
7. `also`
8. `either`
9. `~` (ternary)

10. |> (pipe)

String Operations

sdev has a rich string library. All string functions return new strings — strings are immutable.

```
forge s be "  Hello, World!  "

// Case conversion
speak(upper(s))          // "  HELLO, WORLD!  "
speak(lower(s))          // "  hello, world!  "

// Trimming
speak(trim(s))           // "Hello, World!"

// Length
speak(measure(s))        // 18

// Contains / starts / ends
speak(contains(s, "World"))      // yep
speak(startswith(trim(s), "Hello"))  // yep
speak(endswith(trim(s), "!"))        // yep

// Replace (replaces ALL occurrences)
speak(replace("banana", "a", "o"))   // "bonono"

// Find index
speak(locate("hello world", "world"))  // 6
speak(locate("hello", "xyz"))          // -1

// Split and join
forge parts be shatter("a,b,c,d", ",")
speak(parts)                 // ["a", "b", "c", "d"]
speak(weave(parts, " | "))   // "a | b | c | d"
```

```
// Character list
speak(chars("abc"))   // ["a", "b", "c"]

// Reverse
speak(reverse("hello"))  // "olleh"

// Padding
speak(padLeft("5", 4, "0"))    // "0005"
speak(padRight("hi", 6, "!"))  // "hi!!!!"

// Format strings (use {} as placeholders)
speak(format("Hello, {}! You are {} years old.", "Bob", 25))
// "Hello, Bob! You are 25 years old."

// Repeat
speak(repeat("ab", 3))   // "ababab"

// Substring
speak(snatch("hello world", 6, 11))  // "world"
```

Control Flow

If / Else — `ponder` / `otherwise`

```
forge score be 87

ponder score &gt;= 90 ::
  speak("Grade: A")
;; otherwise ponder score &gt;= 80 ::
  speak("Grade: B")
;; otherwise ponder score &gt;= 70 ::
  speak("Grade: C")
;; otherwise ponder score &gt;= 60 ::
  speak("Grade: D")
;; otherwise ::
```

```
  speak("Grade: F")
;;
```

Inline (for short bodies):

```
ponder x &gt; 0 :: speak("positive") ;; otherwise :: speak("non-pos
```

While Loop — `cycle`

```
forge n be 1
cycle n &lt;= 10 ::
  speak(n)
  n be n + 1
;;

// Infinite loop with break
cycle yep ::
  forge x be randint(1, 10)
  speak("Got:", x)
  ponder x equals 7 ::
    speak("Found 7! Stopping.")
    yeet    // break
  ;;
;;
```

For-Each Loop — `iterate through`

```
forge fruits be ["apple", "banana", "cherry"]

iterate fruit through fruits ::
  speak(fruit)
;;
// apple
// banana
// cherry
```

For-In Loop — `within`

```
// Iterate over a list
within fruit be ["apple", "banana", "cherry"] ::
  speak(fruit)
;;

// Iterate over a range
within i be sequence(5) ::
  speak(i)   // 0 1 2 3 4
;;

within i be sequence(1, 11) ::
  speak(i)   // 1 2 3 ... 10
;;

within i be sequence(0, 20, 5) ::
  speak(i)   // 0 5 10 15
;;
```

Loop Control

`yeet` — Break

Exit the nearest enclosing loop immediately:

```
within i be sequence(100) ::
  ponder i &gt; 5 ::
    yeet
  ;;
  speak(i)
;;
// 0 1 2 3 4 5
```

`skip` — Continue

Skip the rest of the current iteration and go to the next:

```
within i be sequence(10) ::
  ponder i % 2 equals 0 ::
    skip   // skip even numbers
  ;;
  speak(i)   // prints only odd: 1 3 5 7 9
;;
```

Nested Loops

```
within i be sequence(1, 4) ::
  within j be sequence(1, 4) ::
    speak(i + " x " + j + " = " + morph(i * j, "text"))
  ;;
;;
```

Functions

Basic Functions — `conjure`

```
conjure greet(name) ::
  speak("Hello, " + name + "!")
;;

greet("Alice")
greet("World")
```

Return Values — `yield`

```
conjure add(a, b) ::
  yield a + b
```

```
;;

conjure max(a, b) ::
  ponder a &gt; b :: yield a ;;
  yield b
;;

forge sum be add(10, 20)
speak(sum)         // 30
speak(max(5, 9))   // 9
```

Default Parameters

```
conjure greet(name, greeting) ::
  ponder greeting equals void :: greeting be "Hello" ;;
  yield greeting + ", " + name + "!"
;;

speak(greet("Alice"))           // Hello, Alice!
speak(greet("Bob", "Welcome"))  // Welcome, Bob!
```

Lambda Functions (Arrow Syntax)

Single expression, one parameter:

```
forge double be x -&gt; x * 2
forge square be x -&gt; x ^ 2
forge negate be x -&gt; -x

speak(double(5))   // 10
speak(square(4))   // 16
speak(negate(7))   // -7
```

Multiple parameters (parentheses required):

```
forge add be (a, b) -&gt; a + b
```

```
forge clamp be (v, lo, hi) -&gt; v &lt; lo ~ lo : v &gt; hi ~ hi :

speak(add(3, 4))           // 7
speak(clamp(150, 0, 100))  // 100
```

Multi-statement lambda body with `::` and `;;`:

```
forge process be x -&gt; ::
  forge doubled be x * 2
  forge result be doubled + 10
  yield result
;;

speak(process(5))   // 20
```

Closures

Functions capture variables from their enclosing scope:

```
conjure makeCounter(start) ::
  forge count be start

  conjure increment() ::
    count be count + 1
    yield count
  ;;

  conjure reset() ::
    count be start
  ;;

  yield :: "next": increment, "reset": reset ;;
;;

forge c be makeCounter(0)
speak(c.next())   // 1
speak(c.next())   // 2
```

```
speak(c.next())   // 3
c.reset()
speak(c.next())   // 1
```

Recursive Functions

```
conjure factorial(n) ::
  ponder n &lt;= 1 :: yield 1 ;;
  yield n * factorial(n - 1)
;;

speak(factorial(10))   // 3628800

// Mutual recursion
conjure isEven(n) ::
  ponder n equals 0 :: yield yep ;;
  yield isOdd(n - 1)
;;

conjure isOdd(n) ::
  ponder n equals 0 :: yield nope ;;
  yield isEven(n - 1)
;;

speak(isEven(4))   // yep
speak(isOdd(7))    // yep
```

Higher-Order Functions as Parameters

```
conjure applyTwice(f, x) ::
  yield f(f(x))
;;

forge double be x -&gt; x * 2
speak(applyTwice(double, 3))   // 12

conjure compose(f, g) ::
```

```
  yield x -&gt; f(g(x))
;;

forge addOne be x -&gt; x + 1
forge triple be x -&gt; x * 3
forge addOneThenTriple be compose(triple, addOne)
speak(addOneThenTriple(4))   // 15
```

Variadic-style Functions

Use lists to simulate variadic arguments:

```
conjure sumAll(nums) ::
  yield fold(nums, 0, (acc, x) -&gt; acc + x)
;;

speak(sumAll([1, 2, 3, 4, 5, 6, 7, 8, 9, 10]))   // 55
```

Object-Oriented Programming

sdev has full class-based OOP using `essence` for class definitions, `new` for instantiation, `self` for the instance reference, `super` for parent class access, and `extend` for inheritance.

Defining a Class — `essence`

```
essence Person ::
  conjure init(self, name, age) ::
    self.name be name
```

```
    self.age be age
  ;;

  conjure greet(self) ::
    speak("Hi, I'm " + self.name + " and I'm " + morph(self.age, "
  ;;

  conjure birthday(self) ::
    self.age be self.age + 1
    speak(self.name + " is now " + morph(self.age, "text") + "!")
  ;;

  conjure toString(self) ::
    yield "Person(" + self.name + ", " + morph(self.age, "text") +
  ;;
;;
```

Creating Instances — `new`

```
forge alice be new Person("Alice", 30)
forge bob be new Person("Bob", 25)

alice.greet()   // Hi, I'm Alice and I'm 30 years old.
bob.greet()     // Hi, I'm Bob and I'm 25 years old.

alice.birthday()   // Alice is now 31!
speak(alice.age)   // 31

speak(alice.toString())  // Person(Alice, 31)
```

Inheritance — `extend`

```
essence Animal ::
  conjure init(self, name, sound) ::
    self.name be name
    self.sound be sound
    self.energy be 100
```

```
    ;;

    conjure makeSound(self) ::
      speak(self.name + " says: " + self.sound)
    ;;

    conjure eat(self, food) ::
      self.energy be self.energy + 20
      speak(self.name + " eats " + food)
    ;;

    conjure status(self) ::
      speak(self.name + " | Energy: " + morph(self.energy, "text"))
    ;;
  ;;

  essence Dog extend Animal ::
    conjure init(self, name, breed) ::
      super.init(name, "Woof!")
      self.breed be breed
      self.tricks be []
    ;;

    conjure makeSound(self) ::
      speak(self.name + " barks: WOOF WOOF!")   // Override parent
    ;;

    conjure learnTrick(self, trick) ::
      gather(self.tricks, trick)
      speak(self.name + " learned: " + trick)
    ;;

    conjure showTricks(self) ::
      ponder measure(self.tricks) equals 0 ::
        speak(self.name + " knows no tricks.")
        yield void
      ;;
      speak(self.name + " can do: " + weave(self.tricks, ", "))
    ;;
```

```
;;

forge rex be new Dog("Rex", "German Shepherd")
rex.makeSound()                // Rex barks: WOOF WOOF!
rex.eat("kibble")              // Rex eats kibble
rex.learnTrick("sit")
rex.learnTrick("shake")
rex.learnTrick("roll over")
rex.showTricks()               // Rex can do: sit, shake, roll ove
rex.status()                   // Rex | Energy: 120
```

Multi-level Inheritance

```
essence Vehicle ::
  conjure init(self, brand, speed) ::
    self.brand be brand
    self.speed be speed
    self.fuel be 100
  ;;

  conjure describe(self) ::
    yield self.brand + " going " + morph(self.speed, "text") + " kn
  ;;
;;

essence Car extend Vehicle ::
  conjure init(self, brand, speed, doors) ::
    super.init(brand, speed)
    self.doors be doors
  ;;

  conjure honk(self) ::
    speak(self.brand + ": Beep beep!")
  ;;
;;

essence ElectricCar extend Car ::
  conjure init(self, brand, speed, doors, range) ::
```

```
    super.init(brand, speed, doors)
    self.range be range
    self.battery be 100
  ;;

  conjure charge(self) ::
    self.battery be 100
    speak(self.brand + " fully charged!")
  ;;

  conjure describe(self) ::
    yield super.describe() + " (Electric, " + morph(self.range, "to
  ;;
;;

forge tesla be new ElectricCar("Tesla Model 3", 250, 4, 500)
speak(tesla.describe())   // Tesla Model 3 going 250 km/h (Electric
tesla.honk()              // Tesla Model 3: Beep beep!
tesla.charge()            // Tesla Model 3 fully charged!
```

Properties and Computed Properties

```
essence Circle ::
  conjure init(self, radius) ::
    self.radius be radius
  ;;

  conjure area(self) ::
    yield PI * self.radius ^ 2
  ;;

  conjure circumference(self) ::
    yield 2 * PI * self.radius
  ;;

  conjure scale(self, factor) ::
    self.radius be self.radius * factor
    yield self
```

```
  ;;

  conjure toString(self) ::
    yield "Circle(r=" + morph(nearby(self.radius * 100) / 100, "tex
  ;;
;;

forge c be new Circle(5)
speak(nearby(c.area()))           // 79
speak(nearby(c.circumference()))  // 31

c.scale(2)
speak(c.toString())   // Circle(r=10)
```

Error Handling

Try / Rescue — `attempt` / `rescue`

Use `attempt` to try code that might fail, and `rescue` to catch errors:

```
attempt ::
  forge result be 10 / 0
  speak(result)
;; rescue err ::
  speak("Caught an error: " + err)
;;

attempt ::
  forge nums be [1, 2, 3]
  speak(nums[10])   // index out of bounds
;; rescue err ::
  speak("Error accessing list:", err)
;;
```

Nested Error Handling

```
conjure safeDivide(a, b) ::
  attempt ::
    ponder b equals 0 ::
      speak("Error: division by zero")
      yield void
    ;;
    yield a / b
  ;; rescue err ::
    speak("Unexpected error:", err)
    yield void
  ;;
;;

speak(safeDivide(10, 2))   // 5
speak(safeDivide(10, 0))   // Error: division by zero
```

Assertion

```
conjure validateAge(age) ::
  attempt ::
    ponder age &lt; 0 ::
      speak("Error: Age cannot be negative")
      yield nope
    ;;
    ponder age &gt; 150 ::
      speak("Error: Age is unreasonably large")
      yield nope
    ;;
    yield yep
  ;; rescue err ::
    speak("Validation error:", err)
    yield nope
  ;;
;;
```

Async & Concurrency

Async Functions

Use `async conjure` to define an asynchronous function:

```
async conjure fetchData(url) ::
  forge response be await fetch(url)
  yield response
;;
```

Await

Use `await` inside `async` functions to wait for asynchronous operations:

```
async conjure loadUserProfile(id) ::
  forge user be await fetchData("https://api.example.com/users/" +
  forge posts be await fetchData("https://api.example.com/posts?us

  yield ::
    "user": user,
    "posts": posts
  ;;
;;
```

Spawn (Concurrent Execution)

Use `spawn` to run functions concurrently without waiting for them:

```
conjure worker(name, ms) ::
  delay(ms)
  speak(name + " finished after " + morph(ms, "text") + "ms")
;;

spawn worker("Task A", 2000)
spawn worker("Task B", 500)
spawn worker("Task C", 1000)
// Task B completes first, then C, then A
```

Delay

Pause execution for a number of milliseconds:

```
speak("Starting...")
delay(1000)
speak("One second later...")
delay(2000)
speak("Three seconds total")
```

Built-in Output Functions

| Function | Description | Example |
|---------|----------|---------| | `speak(...)` | Print values with spaces between them | `speak("x =", 42)` → `x = 42` | | `whisper(...)` | Print values concatenated (no spaces) | `whisper("a", "b")` → `ab` | | `shout(...)` | Print values uppercased | `shout("hello")` → `HELLO` |

```
forge x be 42
forge name be "Alice"

speak("The answer is", x)       // The answer is 42
```

```
speak(name, "scored", x, "%")   // Alice scored 42 %
whisper("[", x, "]")             // [42]
shout("warning: low battery")    // WARNING: LOW BATTERY
```

Built-in Math Functions

Core Math

Function	Description	Example
`magnitude(x)`	Absolute value	`magnitude(-5)` → `5`
`root(x)`	Square root	`root(16)` → `4`
`ground(x)`	Floor (round down)	`ground(3.7)` → `3`
`elevate(x)`	Ceiling (round up)	`elevate(3.2)` → `4`
`nearby(x)`	Round to nearest int	`nearby(3.5)` → `4`
`least(...)`	Minimum value	`least(3, 1, 4, 1)` → `1`
`greatest(...)`	Maximum value	`greatest(3, 1, 4, 1)` → `4`

Trigonometry

Function	Description
`sin(x)`	Sine (radians)
`cos(x)`	Cosine (radians)
`tan(x)`	Tangent (radians)
`asin(x)`	Arcsine
`acos(x)`	Arccosine
`atan(x)`	Arctangent
`atan2(y, x)`	Two-argument arctangent
`radians(deg)`	Convert degrees to radians
`degrees(rad)`	Convert radians to degrees

Logarithms & Exponentials

| Function | Description | |----------|----------| | `log(x)` | Natural logarithm | | `log10(x)` | Base-10 logarithm | | `log2(x)` | Base-2 logarithm | | `exp(x)` | e^x |

Advanced Math

| Function | Description | Example | |----------|----------|---------| | `clamp(v, min, max)` | Constrain value to range | `clamp(150, 0, 100)` → `100` | | `lerp(a, b, t)` | Linear interpolation | `lerp(0, 100, 0.3)` → `30` | | `mapRange(v, fromLo, fromHi, toLo, toHi)` | Map value between ranges | `mapRange(50, 0, 100, 0, 1)` → `0.5` | | `sum(list)` | Sum all elements of a list | `sum([1,2,3,4])` → `10` | | `average(list)` | Mean value of a list | `average([1,2,3,4])` → `2.5` | | `sign(x)` | Sign: -1, 0, or 1 | `sign(-5)` → `-1` | | `dist(x1, y1, x2, y2)` | Distance between two points | `dist(0,0,3,4)` → `5` |

Random

| Function | Description | Example | |----------|----------|---------| | `chaos()` | Random float 0-1 | `chaos()` → `0.7234` | | `randint(min, max)` | Random integer (inclusive) | `randint(1, 6)` → `4` | | `pick(list)` | Random element from list | `pick(["a","b","c"])` → `"b"` | | `shuffle(list)` | Shuffle a copy of list | `shuffle([1,2,3])` → `[3,1,2]` |

```
// Dice rolling
```

```
conjure rollDice(sides) ::
  yield randint(1, sides)
;;

speak("d6:", rollDice(6))
speak("d20:", rollDice(20))

// Coin flip
speak(chaos() &gt; 0.5 ~ "Heads" : "Tails")
```

Built-in String Functions

| Function | Description | Signature |
|---------|----------|----------| | `upper(s)` | Uppercase | `upper("hi")` → `"HI"` | | `lower(s)` | Lowercase | `lower("HI")` → `"hi"` | | `trim(s)` | Remove surrounding whitespace | `trim("  hi  ")` → `"hi"` | | `reverse(s)` | Reverse string | `reverse("abc")` → `"cba"` | | `measure(s)` | String length | `measure("hello")` → `5` | | `contains(s, sub)` | Check substring | `contains("hello", "ell")` → `yep` | | `startswith(s, prefix)` | Starts with | `startswith("hello", "he")` → `yep` | | `endswith(s, suffix)` | Ends with | `endswith("hello", "lo")` → `yep` | | `replace(s, old, new)` | Replace all occurrences | `replace("aaa", "a", "b")` → `"bbb"` | | `locate(s, sub)` | Index of substring (-1 if not found) | `locate("hello", "l")` → `2` | | `shatter(s, sep)` | Split string into list | `shatter("a,b", ",")` → `["a","b"]` | | `chars(s)` | Split into list of characters | `chars("abc")` → `["a","b","c"]` | | `weave(list, sep)` | Join list with separator | `weave(["a","b"], "-")` → `"a-b"` | | `padLeft(s, w, c)` | Pad left to width | `padLeft("5", 3, "0")` → `"005"` | |

`padRight(s, w, c)` | Pad right to width | `padRight("hi", 5, ".")` → `"hi..."` | | `format(s, ...)` | Substitute `{}` placeholders | `format("Hi {}!", "Bob")` → `"Hi Bob!"` | | `repeat(s, n)` | Repeat string n times | `repeat("ab", 3)` → `"ababab"` | | `snatch(s, start, end?)` | Substring by index | `snatch("hello", 1, 3)` → `"el"` |

Built-in List Functions

Function	Description	Example
`measure(list)`	Length of list	`measure([1,2,3])` → `3`
`gather(list, item)`	Append item (mutates)	`gather([1,2], 3)` → `[1,2,3]`
`pluck(list)`	Remove and return last item	`pluck([1,2,3])` → `3`
`snatch(list, idx)`	Remove item at index	`snatch([1,2,3], 1)` → removes `2`
`insert(list, idx, item)`	Insert at index	`insert([1,3], 1, 2)` → `[1,2,3]`
`portion(list, start, end?)`	Slice	`portion([1,2,3,4], 1, 3)` → `[2,3]`
`reverse(list)`	Return reversed copy	`reverse([1,2,3])` → `[3,2,1]`
`sort(list)`	Sort ascending copy	`sort([3,1,2])` → `[1,2,3]`
`sortDesc(list)`	Sort descending copy	`sortDesc([1,3,2])` → `[3,2,1]`
`unique(list)`	Remove duplicates	`unique([1,1,2,2,3])` → `[1,2,3]`
`flatten(list)`	Flatten nested lists	`flatten([[1,2],[3,4]])` → `[1,2,3,4]`
`concat(a, b)`	Concatenate two lists	`concat([1,2],[3,4])` → `[1,2,3,4]`
`contains(list, item)`	Check membership	`contains([1,2,3], 2)` → `yep`
`first(list)`	First element

| `first([1,2,3])` → `1` | | `last(list)` | Last element | `last([1,2,3])` → `3` | | `clone(list)` | Deep copy | `clone([1,[2,3]])` |

Built-in Tome (Dict) Functions

| Function | Description | |----------|----------| | `inscriptions(tome)` | Get all keys as a list | | `contents(tome)` | Get all values as a list | | `entries(tome)` | Get `[[key, value], ...]` pairs | | `contains(tome, key)` | Check if key exists | | `merge(t1, t2, ...)` | Merge dicts (later overrides earlier) | | `erase(tome, key)` | Remove key |

```
forge a be :: "x": 1, "y": 2 ;;
forge b be :: "y": 99, "z": 3 ;;
forge c be merge(a, b)
speak(c)  // {"x": 1, "y": 99, "z": 3}
```

Input / Output

`input(prompt?)` — Read User Input

Prompts the user for input and returns the entered text:

```
forge name be input("What is your name? ")
speak("Hello, " + name + "!")
```

```
forge age be morph(input("Enter your age: "), "number")
speak("You are " + age + " years old")
```

`print(...)` / `println(...)` — Print (Aliases)

Standard aliases for `speak()`:

```
print("Hello")
println("World")
```

Character & Code Point Functions

Function	Description	Example
`chr(n)`	Number to character	`chr(65)` → `"A"`
`ord(c)`	Character to number	`ord("A")` → `65`

```
speak(chr(72) + chr(105))  // "Hi"
speak(ord("A"))             // 65

// Build a Caesar cipher
conjure encrypt(text, shift) ::
  forge result be ""
  iterate through text ::
    forge code be ord(item) + shift
    result be result + chr(code)
  ;;
  yield result
;;
speak(encrypt("ABC", 3))  // "DEF"
```

Number Base Conversion

| Function | Description | Example | |---------|----------|---------| | `hex(n)` | Number to hex string | `hex(255)` → `"0xFF"` | | `oct(n)` | Number to octal | `oct(8)` → `"0o10"` | | `bin(n)` | Number to binary | `bin(10)` → `"0b1010"` | | `parseNum(str, base?)` | Parse string with base | `parseNum("FF", 16)` → `255` |

```
speak(hex(255))        // "0xFF"
speak(bin(42))         // "0b101010"
speak(oct(64))         // "0o100"
speak(parseNum("1010", 2))  // 10
speak(parseNum("FF", 16))   // 255
```

Number Formatting & Checking

| Function | Description | Example | |---------|----------|---------| | `toFixed(n, digits)` | Format decimal places | `toFixed(3.14159, 2)` → `"3.14"` | | `toPrecision(n, p)` | Format to precision | `toPrecision(123.456, 4)` → `"123.5"` | | `isNaN(v)` | Check if NaN | `isNaN(0/0)` → `yep` | | `isFinite(v)` | Check if finite | `isFinite(INFINITY)` → `nope` | | `isInteger(v)` | Check if integer | `isInteger(3.0)` → `yep` |

```
forge pi be 3.14159265
speak(toFixed(pi, 2))      // "3.14"
speak(toPrecision(pi, 4))  // "3.142"
```

```
speak(isInteger(42))       // yep
speak(isFinite(1/0))       // nope
```

String Checking Functions

| Function | Description | Example |
|---------|-----------|---------| | `capitalize(s)` | First char uppercase | `capitalize("hello")` → `"Hello"` | | `title(s)` | Title Case | `title("hello world")` → `"Hello World"` | | `center(s, width, char?)` | Center-pad | `center("hi", 10, "-")` → `"----hi----"` | | `trimLeft(s)` | Trim left whitespace | `trimLeft("  hi")` → `"hi"` | | `trimRight(s)` | Trim right whitespace | `trimRight("hi  ")` → `"hi"` | | `isUpper(s)` | All uppercase? | `isUpper("ABC")` → `yep` | | `isLower(s)` | All lowercase? | `isLower("abc")` → `yep` | | `isDigit(s)` | All digits? | `isDigit("123")` → `yep` | | `isAlpha(s)` | All alphabetic? | `isAlpha("abc")` → `yep` | | `isAlphaNum(s)` | All alphanumeric? | `isAlphaNum("abc123")` → `yep` | | `isSpace(s)` | All whitespace? | `isSpace(" ")` → `yep` |

```
speak(capitalize("hello world"))  // "Hello world"
speak(title("the quick brown fox"))  // "The Quick Brown Fox"
speak(center("TITLE", 20, "="))  // "=======TITLE========"
speak(isDigit("42"))   // yep
speak(isAlpha("hello"))  // yep
```

Regex / Pattern Matching

| Function | Description | |----------|----------| | `match(text, pattern)` | First regex match (returns list or void) | | `matchAll(text, pattern)` | All regex matches | | `replaceRegex(text, pattern, replacement)` | Regex replace (global) | | `test(text, pattern)` | Test if pattern matches |

```
forge text be "Hello 123 World 456"
speak(match(text, "\\d+"))           // ["123"]
speak(matchAll(text, "\\d+"))        // [["123"], ["456"]]
speak(replaceRegex(text, "\\d+", "#"))  // "Hello # World #"
speak(test(text, "\\d+"))           // yep

// Validate email
conjure isEmail(s) ::
  yield test(s, "^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\\.[a-zA-Z]{2,}$
;;
speak(isEmail("user@example.com"))  // yep
```

Bitwise Operations

| Function | Description | Example | |----------|----------|---------| | `bitAnd(a, b)` | Bitwise AND | `bitAnd(5, 3)` → `1` | | `bitOr(a, b)` | Bitwise OR | `bitOr(5, 3)` → `7` | | `bitXor(a, b)` | Bitwise XOR | `bitXor(5, 3)` → `6` | | `bitNot(a)` | Bitwise NOT | `bitNot(0)` → `-1` | | `bitShiftLeft(a, n)` | Left shift | `bitShiftLeft(1, 3)` → `8` | | `bitShiftRight(a, n)` | Right shift | `bitShiftRight(8, 2)` → `2` |

```
// Flags / bitmask example
forge READ be 1
```

```
forge WRITE be 2
forge EXEC be 4

forge perms be bitOr(READ, WRITE)  // 3
speak(bitAnd(perms, READ) &gt; 0)     // yep (has read)
speak(bitAnd(perms, EXEC) &gt; 0)     // nope (no exec)
```

Base64 Encoding

| Function | Description | |----------|----------| | `base64encode(text)` | Encode text to base64 | | `base64decode(text)` | Decode base64 to text |

```
forge encoded be base64encode("Hello, World!")
speak(encoded)  // "SGVsbG8sIFdvcmxkIQ=="
speak(base64decode(encoded))  // "Hello, World!"
```

Hash Function

```
speak(hash("hello"))     // deterministic 32-bit integer hash
speak(hash([1, 2, 3]))   // works on any value
```

Time & Date

| Function | Description | |----------|----------| | `now()` | Current timestamp in milliseconds | | `timestamp()` | ISO 8601 string | | `time()` | Detailed time tome with year, month, day, etc. | | `formatTime(ms)` | Format milliseconds

to ISO string |

```
forge t be time()
speak("Year: " + t.year)
speak("Month: " + t.month)
speak("Day: " + t.day)
speak("Hour: " + t.hour)
speak("ISO: " + t.iso)

forge start be now()
// ... do work ...
forge elapsed be now() - start
speak("Took " + elapsed + "ms")
```

Functional Programming

| Function | Description | |----------|----------| | `compose(f, g, ...)` | Right-to-left function composition | | `pipe(value, f, g, ...)` | Left-to-right value piping | | `curry(fn, arity)` | Currying | | `memoize(fn)` | Cache function results | | `tap(value, fn)` | Execute fn with value, return value | | `times(n, fn)` | Call fn n times with index | | `groupBy(list, fn)` | Group elements by key function | | `chunk(list, size)` | Split list into chunks |

```
// Compose
conjure double(x) :: yield x * 2 ;;
conjure addOne(x) :: yield x + 1 ;;
forge doubleAndAdd be compose(addOne, double)
speak(doubleAndAdd(5))  // 11

// Pipe
forge result be pipe(5, double, addOne, double)
```

```
speak(result)  // 22

// Memoize (cache expensive computations)
conjure fib(n) ::
  ponder n &lt;= 1 :: yield n ;;
  yield fib(n - 1) + fib(n - 2)
;;
forge fastFib be memoize(fib)
speak(fastFib(10))  // 55

// Group and chunk
forge words be ["apple", "avocado", "banana", "blueberry", "cherry
speak(groupBy(words, w -&gt; charAt(w, 0)))
// :: a: ["apple", "avocado"], b: ["banana", "blueberry"], c: ["che

speak(chunk([1,2,3,4,5,6,7], 3))  // [[1,2,3], [4,5,6], [7]]

// Times
speak(times(5, i -&gt; i * i))  // [0, 1, 4, 9, 16]
```

Buffer / Memory Operations

sdev provides low-level byte buffer operations for systems programming:

`buffer(size)` — Create Byte Buffer

```
forge mem be buffer(1024)
mem.set(0, 255)
mem.set(1, 128)
speak(mem.get(0))    // 255
speak(mem.get(1))    // 128
speak(mem.size())    // 1024
```

Buffer Methods

| Method | Description | |--------|----------| | `buf.get(index)` | Read byte at index | | `buf.set(index, value)` | Write byte at index (0-255) | | `buf.fill(value)` | Fill entire buffer | | `buf.slice(start, end)` | Get portion as list | | `buf.toList()` | Convert to list of numbers | | `buf.toText()` | Decode as UTF-8 text | | `buf.fromString(text)` | Write UTF-8 text into buffer | | `buf.copyTo(target)` | Copy data to another buffer | | `buf.size()` | Get buffer size |

`pointer(buffer, offset)` — Memory Pointer

```
forge mem be buffer(256)
forge ptr be pointer(mem, 0)
ptr.write(42)
speak(ptr.read())        // 42

forge ptr2 be ptr.advance(4)
ptr2.writeU16(1024)
speak(ptr2.readU16())    // 1024

ptr2.writeU32(0xDEADBEEF)
speak(hex(ptr2.readU32()))  // "0xDEADBEEF"
```

Pointer Methods

| Method | Description | |--------|----------| | `ptr.read()` | Read byte | | `ptr.write(v)` | Write byte | | `ptr.advance(n?)` | Return new pointer at offset+n | | `ptr.readU16()` | Read 16-bit unsigned (little-endian) | | `ptr.readU32()` | Read 32-bit unsigned (little-endian) | | `ptr.writeU16(v)` | Write

16-bit unsigned | | `ptr.writeU32(v)` | Write 32-bit unsigned |

Error Handling & Control Flow

| Function | Description | |----------|----------| | `exit(code?)` | Terminate program with exit code | | `panic(message)` | Fatal error (kernel panic) | | `throw(message)` | Throw custom error |

```
// Throw and catch
attempt ::
  forge x be input("Enter a number: ")
  forge n be morph(x, "number")
  ponder n &lt; 0 ::
    throw("Negative numbers not allowed!")
  ;;
  speak("Square root: " + root(n))
rescue err ::
  speak("Error: " + err)
;;

// Exit
ponder badCondition ::
  exit(1)
;;
```

Additional Aliases

| Alias | Original | Description | |-------|----------|-----------| | `print()` | `speak()` | Standard print | | `range()` | `sequence()` | Standard range | | `typeof()` | `gettype()` | Type checking | | `sleep()` | `delay()` | Pause (no-op in browser) | | `keys()` | `inscriptions()` | Dict keys | | `values()` | `contents()` | Dict values | | `freeze(obj)` | — | Make object immutable | | `isFrozen(obj)` | — | Check if frozen | | `hash(value)` | — | 32-bit hash code |

Higher-Order Functions

`each` — Transform (Map)

Apply a function to every element and return a new list:

```
forge nums be [1, 2, 3, 4, 5]
speak(each(nums, x -&gt; x ^ 2))          // [1, 4, 9, 16, 25]
speak(each(nums, (x, i) -&gt; x + i))     // [1, 3, 5, 7, 9]  (valu
speak(each(["a","b","c"], upper))       // ["A", "B", "C"]
```

`sift` — Filter

Keep only elements where the predicate returns `yep`:

```
forge nums be [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
speak(sift(nums, x -&gt; x % 2 equals 0))       // [2, 4, 6, 8, 10
speak(sift(nums, x -&gt; x &gt; 5))                // [6, 7, 8, 9,
speak(sift(["", "hi", "", "world"], x -&gt; measure(x) &gt; 0))  /
```

`fold` — Reduce

Reduce a list to a single value using an accumulator:

```
forge nums be [1, 2, 3, 4, 5]
speak(fold(nums, 0, (acc, x) -&gt; acc + x))   // 15 (sum)
speak(fold(nums, 1, (acc, x) -&gt; acc * x))   // 120 (product)

// Build a string
speak(fold(["a","b","c"], "", (acc, x) -&gt; acc + x + "-"))  // "a

// Find max
speak(fold(nums, nums[0], (a, b) -&gt; a &gt; b ~ a : b))   // 5
```

`seek` — Find First

Return the first element matching a predicate, or `void`:

```
forge people be [
  :: "name": "Alice", "age": 22 ;;,
  :: "name": "Bob", "age": 35 ;;,
  :: "name": "Carol", "age": 28 ;;
]

forge adult be seek(people, p -&gt; p.age &gt;= 30)
speak(adult.name)   // Bob

forge nobody be seek(people, p -&gt; p.age &gt; 100)
speak(nobody)   // void
```

`every` — All Match

Returns `yep` if ALL elements satisfy the predicate:

```
speak(every([2,4,6,8], x -&gt; x % 2 equals 0))  // yep
speak(every([1,2,3], x -&gt; x &gt; 0))             // yep
speak(every([1,-2,3], x -&gt; x &gt; 0))            // nope
```

`some` — Any Match

Returns `yep` if ANY element satisfies the predicate:

```
speak(some([1,3,5,6], x -&gt; x % 2 equals 0))  // yep (6 is even)
speak(some([1,3,5,7], x -&gt; x % 2 equals 0))  // nope
```

`enumerate` — Index + Value

Pairs each element with its index:

```
speak(enumerate(["a","b","c"]))
// [[0,"a"], [1,"b"], [2,"c"]]

within pair be enumerate(["red","green","blue"]) ::
  speak(pair[0] + ": " + pair[1])
;;
// 0: red
// 1: green
// 2: blue
```

`zip` — Pair Two Lists

Combine two lists element-by-element:

```
forge keys be ["name", "age", "city"]
forge vals be ["Alice", 30, "NYC"]
speak(zip(keys, vals))
// [["name","Alice"], ["age",30], ["city","NYC"]]
```

Type System & Conversion

Getting Type — `essence()`

Returns the type name as a string:

```
speak(essence(42))          // "number"
speak(essence("hi"))        // "text"
speak(essence(yep))         // "truth"
speak(essence(void))        // "void"
speak(essence([1,2,3]))     // "list"
speak(essence(::;;))        // "tome"
```

Converting Types — `morph()`

```
speak(morph("42", "number"))     // 42
speak(morph(42, "text"))         // "42"
speak(morph(1, "truth"))         // yep
speak(morph(0, "truth"))         // nope
speak(morph("3.14", "number"))   // 3.14
```

Type Checking Helpers

```
speak(essence(x) equals "number")      // Check if number
speak(essence(x) equals "text")        // Check if string
speak(essence(x) equals "list")        // Check if list
speak(essence(x) equals "tome")        // Check if dict
speak(x equals void)                   // Check if null
```

Collections

Set

A collection of unique elements. Duplicate additions are silently ignored.

```
forge s be Set()
s.add(1)
s.add(2)
s.add(2)    // Duplicate ignored
s.add(3)

speak(s.size())     // 3
speak(s.has(2))     // yep
speak(s.has(99))    // nope
speak(s.values())   // [1, 2, 3]

s.remove(2)
speak(s.values())   // [1, 3]

s.clear()
speak(s.size())     // 0
speak(s.isEmpty())  // yep
```

Set methods: `add(v)`, `remove(v)`, `has(v)`, `values()`, `size()`, `isEmpty()`, `clear()`, `toList()`

Map

A key-value store where keys can be any type (including objects):

```
forge m be Map()
m.set("name", "Alice")
m.set("age", 30)
m.set("active", yep)

speak(m.get("name"))     // Alice
speak(m.has("age"))      // yep
```

```
speak(m.keys())          // ["name", "age", "active"]
speak(m.values())        // ["Alice", 30, yep]
speak(m.entries())       // [["name","Alice"], ["age",30], ["activ
speak(m.size())          // 3

m.delete("active")
speak(m.size())          // 2
m.clear()
```

Map methods: `set(k, v)`, `get(k)`, `has(k)`, `delete(k)`, `keys()`, `values()`, `entries()`, `size()`, `isEmpty()`, `clear()`

Queue (FIFO)

First-in, first-out data structure:

```
forge q be Queue()
q.enqueue("first")
q.enqueue("second")
q.enqueue("third")

speak(q.peek())      // first  (look without removing)
speak(q.dequeue())   // first  (remove and return)
speak(q.peek())      // second
speak(q.size())      // 2
speak(q.isEmpty())   // nope
```

Queue methods: `enqueue(v)`, `dequeue()`, `peek()`, `size()`, `isEmpty()`, `clear()`, `toList()`

Stack (LIFO)

Last-in, first-out data structure:

```
forge s be Stack()
s.push(10)
```

```
s.push(20)
s.push(30)

speak(s.peek())   // 30  (look without removing)
speak(s.pop())    // 30  (remove and return)
speak(s.pop())    // 20
speak(s.size())   // 1

// Stack-based undo system
forge history be Stack()
history.push("action1")
history.push("action2")
history.push("action3")

speak("Undoing:", history.pop())  // action3
speak("Undoing:", history.pop())  // action2
```

Stack methods: `push(v)`, `pop()`, `peek()`, `size()`, `isEmpty()`, `clear()`, `toList()`

LinkedList

A doubly-linked list with O(1) front/back operations:

```
forge list be LinkedList()
list.append(1)
list.append(2)
list.append(3)
list.prepend(0)

speak(list.toList())   // [0, 1, 2, 3]
speak(list.size())     // 4
speak(list.get(2))     // 2

list.remove(1)         // Remove element with value 1
speak(list.toList())   // [0, 2, 3]

speak(list.head())     // 0
```

```
speak(list.tail())     // 3
```

LinkedList methods: `append(v)`, `prepend(v)`, `remove(v)`, `get(idx)`, `size()`, `head()`, `tail()`, `toList()`, `clear()`

Matrix Operations

sdev has a comprehensive built-in matrix math library useful for data science and machine learning.

Creating Matrices

```
// Matrix filled with a value
forge zeros be matrix(3, 3, 0)
// [[0,0,0], [0,0,0], [0,0,0]]

forge ones be matrix(2, 4, 1)
// [[1,1,1,1], [1,1,1,1]]

// Identity matrix
forge eye be identity(4)
// [[1,0,0,0], [0,1,0,0], [0,0,1,0], [0,0,0,1]]

// Reshape a flat list into a matrix
forge m be reshape([1,2,3,4,5,6], 2, 3)
// [[1,2,3], [4,5,6]]
```

Matrix Arithmetic

```
forge a be [[1, 2], [3, 4]]
forge b be [[5, 6], [7, 8]]

// Element-wise operations
```

```
speak(matadd(a, b))     // [[6,8], [10,12]]
speak(matsub(a, b))     // [[-4,-4], [-4,-4]]
speak(matscale(a, 2))   // [[2,4], [6,8]]

// Matrix multiplication
speak(matmul(a, b))
// [[1*5+2*7, 1*6+2*8], [3*5+4*7, 3*6+4*8]]
// = [[19,22], [43,50]]

// Transpose
speak(transpose(a))
// [[1,3], [2,4]]
```

Vector Operations

```
forge v1 be [1, 2, 3]
forge v2 be [4, 5, 6]

// Dot product
speak(dot(v1, v2))   // 1*4 + 2*5 + 3*6 = 32

// Element-wise multiply via each
forge elem be each(v1, (x, i) -&gt; x * v2[i])
speak(elem)   // [4, 10, 18]
```

Matrix Utilities

| Function | Description | |----------|----------| | `matrix(rows, cols, fill)` | Create filled matrix | | `identity(n)` | Create n×n identity matrix | | `transpose(m)` | Transpose rows and columns | | `matmul(a, b)` | Matrix multiplication | | `matadd(a, b)` | Element-wise addition | | `matsub(a, b)` | Element-wise subtraction | | `matscale(m, s)` | Scalar multiplication | | `dot(a, b)` | Dot product of

two vectors | | `reshape(list, rows, cols)` | Reshape list to 2D matrix | | `flatten(m)` | Flatten 2D matrix to 1D list | | `shape(m)` | Get `[rows, cols]` of matrix | | `sum(m)` | Sum all elements | | `mean(list)` | Mean of a 1D list |

Neural Network Example

```
// Sigmoid activation function
conjure sigmoid(x) ::
  yield 1 / (1 + exp(-x))
;;

// Feedforward neural network layer
conjure layer(inputs, weights, biases) ::
  // inputs: [1 x n], weights: [n x m], biases: [1 x m]
  forge z be matmul(inputs, weights)
  forge zb be matadd(z, biases)
  // Apply sigmoid element-wise
  yield each(zb, row -&gt; each(row, x -&gt; sigmoid(x)))
;;

forge input be [[0.5, 0.3, 0.2]]
forge w1 be [[0.1, 0.2], [0.3, 0.4], [0.5, 0.6]]
forge b1 be [[0.1, 0.1]]

forge output be layer(input, w1, b1)
speak("Layer output:", output)
```

Graphics & Game Development

sdev has a built-in 2D graphics and turtle graphics API. Graphics commands render to the Canvas panel in the IDE.

Canvas Setup

```
canvas(800, 600)    // Set canvas dimensions (width, height)
clear("#1a1a2e")    // Clear with a background color
```

Fill and Stroke

```
fill("blue")          // Set fill color
noFill()              // Disable fill
stroke("red", 2)      // Set stroke: color, width
noStroke()            // Disable stroke
lineWidth(3)          // Set line width separately
alpha(0.5)            // Set global transparency (0–1)
lineCap("round")      // Line cap: "round", "square", "butt"
lineJoin("round")     // Line join: "round", "bevel", "miter"
```

Basic Shapes

```
// Rectangle: rect(x, y, width, height, cornerRadius?)
fill("steelblue")
rect(50, 50, 200, 100)
rect(300, 50, 200, 100, 20)    // Rounded corners

// Circle: circle(x, y, radius)
fill("tomato")
circle(150, 300, 60)

// Ellipse: ellipse(x, y, radiusX, radiusY, rotation?)
fill("gold")
ellipse(350, 300, 100, 50)
```

```
ellipse(550, 300, 100, 50, 45)   // Rotated 45°

// Line: line(x1, y1, x2, y2)
stroke("white", 2)
line(50, 400, 750, 400)

// Arc: arc(x, y, radius, startAngle, endAngle, counterclockwise?)
stroke("lime", 3)
arc(200, 500, 50, 0, 270)

// Triangle: triangle(x1, y1, x2, y2, x3, y3)
fill("orchid")
triangle(400, 450, 450, 550, 350, 550)

// Polygon: polygon([[x,y], ...])
fill("coral")
polygon([[600, 400], [650, 430], [640, 490], [560, 490], [550, 430

// Star: star(x, y, outerR, innerR, points?)
fill("gold")
star(150, 150, 50, 25)          // 5-pointed star
star(300, 150, 60, 25, 8)       // 8-pointed star

// Heart: heart(x, y, size)
fill("red")
heart(500, 150, 50)

// Point: point(x, y, size?)
fill("white")
point(100, 100)
point(150, 100, 8)
```

Text Drawing

```
fill("white")
text("Hello, World!", 100, 100)           // Basic text
text("Large Text", 100, 150, 36)          // With size
text("Bold Text", 100, 200, 24, "bold")   // With style
```

```
font("Arial", "bold")                    // Set font family and s
textAlign("center", "middle")            // Horizontal: left/cente
```

Gradients

```
// Linear gradient: fill with gradient from (x1,y1) to (x2,y2)
// Color stops: [position, color] where position is 0.0 to 1.0
linearGradient(0, 0, 400, 0, [0, "red"], [0.5, "yellow"], [1, "gree
rect(0, 0, 400, 100)

// Radial gradient: from inner circle to outer circle
radialGradient(200, 200, 0, 200, 200, 150, [0, "white"], [0.5, "cya
circle(200, 200, 150)
```

Shadows

```
shadow("rgba(0,0,0,0.5)", 15, 5, 5)   // color, blur, offsetX, offs
fill("gold")
circle(200, 200, 80)
noShadow()
```

Transformations

```
save()                // Push current state
translate(200, 200)   // Move origin to (200, 200)
rotate(0.785)         // Rotate 45° (in radians)
scale(1.5)            // Scale uniformly
rect(-50, -50, 100, 100)   // Draw centered rectangle
restore()             // Pop saved state

resetTransform()      // Reset all transforms to identity
```

Path Drawing

```
// Custom polygon path
fill("purple")
beginPath()
moveTo(100, 100)
lineTo(200, 80)
lineTo(250, 150)
lineTo(200, 220)
lineTo(100, 200)
closePath()
fillPath()
strokePath()

// Bezier curve
stroke("cyan", 2)
beginPath()
moveTo(100, 300)
bezierTo(150, 200, 250, 400, 300, 300)   // cp1x, cp1y, cp2x, cp2y
strokePath()

// Quadratic curve
beginPath()
moveTo(100, 400)
quadraticTo(200, 350, 300, 400)           // cpx, cpy, x, y
strokePath()
```

Turtle Graphics

The turtle graphics API lets you draw by commanding a turtle that moves and draws lines as it goes.

```
canvas(500, 500)
clear("#0d0d20")
turtle()           // Create turtle at center, facing up

// Basic movement
forward(100)       // Move forward 100 units (drawing as it goes)
right(90)          // Turn right 90 degrees
forward(100)
```

```
right(90)
forward(100)
right(90)
forward(100)       // This draws a square!

// Pen control
penup()            // Lift pen — move without drawing
goto(250, 250)     // Jump to coordinates
pendown()          // Lower pen — start drawing again

// Pen style
pencolor("lime")   // Set pen color (any CSS color)
penwidth(3)        // Set pen width

// Advanced
setheading(45)     // Set absolute direction (degrees)
speak(heading())   // Get current heading
speak(pos())       // Get current [x, y] position
home()             // Return to start position

// Turtle circle
turtleCircle(50)        // Draw full circle
turtleCircle(50, 72)    // Polygon approximation (72 steps)

// Dot and stamp
dot(10, "red")     // Draw a filled dot
stamp()            // Leave a mark of the turtle shape

// EXAMPLE: Psychedelic spiral
canvas(600, 600)
clear("#000000")
turtle()
within i be sequence(0, 300) ::
  pencolor(hue(i * 1.2))
  penwidth(1 + i * 0.02)
  forward(i)
  right(91)
;;
```

Sprites & Game Objects

```
// Create sprite: createSprite(x, y, width, height, color?)
forge player be createSprite(100, 100, 40, 40, "blue")
forge enemy be createSprite(400, 300, 40, 40, "red")

// Access/set properties
player.x be 150
player.velocityX be 5
player.velocityY be -2

// Update position (applies velocity)
updateSprite(player)

// Draw sprite
drawSprite(player)
drawSprite(enemy)

// Collision detection
ponder spriteCollides(player, enemy) ::
  speak("HIT!")
;;

// Move by delta
moveSprite(player, 10, 5)
```

Color Functions

```
// CSS color names work directly
fill("red")
fill("steelblue")
fill("rgba(255, 0, 0, 0.5)")

// Helper functions
forge red be rgb(255, 0, 0)
forge transparent be rgba(0, 255, 0, 0.3)

// HSL color — hue(hue, saturation?, lightness?)
```

```
forge cyan be hue(180)              // Cyan (s=100, l=50 defaults)
forge purple be hue(270, 80, 60)    // Custom HSL
forge faded be hsla(240, 100, 50, 0.4)

// Rainbow cycle (great for spirals!)
within i be sequence(360) ::
  fill(hue(i))
  rect(i, 0, 1, 50)
;;

// Random color
fill(randomColor())
```

Graphics Math

```
// Convert degrees/radians
speak(radians(180))   // 3.14159...
speak(degrees(PI))    // 180

// Lerp (smooth transitions)
forge pos be lerp(0, 100, 0.25)   // 25

// Map value between ranges
forge brightness be mapRange(50, 0, 100, 0, 255)   // 127.5

// Clamp
forge safe be clamp(150, 0, 100)   // 100

// Distance between two points
speak(dist(0, 0, 3, 4))   // 5
```

Complete 2D Graphics Example — Starfield

```
canvas(800, 600)
```

```
// Create 200 stars
forge stars be each(sequence(200), i -&gt; ::
  "x": randint(0, 800),
  "y": randint(0, 600),
  "r": chaos() * 2 + 0.5,
  "speed": chaos() * 2 + 0.5,
  "bright": chaos() * 0.7 + 0.3
;;)

clear("#000011")

within star be stars ::
  fill(rgba(255, 255, 255, star.bright))
  circle(star.x, star.y, star.r)
;;

// Draw nebula in center
radialGradient(400, 300, 0, 400, 300, 200,
  [0, "rgba(100,50,200,0.4)"],
  [0.5, "rgba(50,100,255,0.2)"],
  [1, "rgba(0,0,0,0)"]
)
circle(400, 300, 200)
```

File I/O

> ***Note:*** *File I/O is available in the Python standalone interpreter and native desktop builds. In the browser-based IDE, these functions are simulated.*

Reading Files

```
// Read entire file as text
```

```
forge content be decipher("data.txt")
speak(content)

// Read and parse JSON
forge config be unetch(decipher("config.json"))
speak(config.version)
speak(config.name)
```

Writing Files

```
// Write text file
inscribe("output.txt", "Hello, World!\n")

// Write JSON with indentation
forge data be :: "score": 100, "name": "Alice", "level": 5 ;;
inscribe("save.json", etch(data))
```

Append to File

```
appendFile("log.txt", "User logged in\n")
appendFile("log.txt", "User viewed dashboard\n")
```

File Utilities

| Function | Description | |----------|----------| | `decipher(path)` | Read file as text | | `inscribe(path, content)` | Write text to file | | `appendFile(path, content)` | Append to file | | `fileExists(path)` | Returns `yep`/`nope` | | `deleteFile(path)` | Delete a file | | `listDir(path)` | Get list of files in directory |

Networking

> ***Note:*** *Network functions are available in async contexts. In the browser they use* `fetch`*. In native builds they use Node.js http.*

HTTP GET

```
async conjure getUsers() ::
  forge data be await fetch("https://jsonplaceholder.typicode.com/u
  yield data
;;
```

HTTP POST

```
async conjure createUser(name, email) ::
  forge response be await fetch("https://api.example.com/users", :
    "method": "POST",
    "headers": :: "Content-Type": "application/json" ;;,
    "body": :: "name": name, "email": email ;;
  ;;)
  yield response
;;
```

Examples & Recipes

FizzBuzz

```
within i be sequence(1, 101) ::
  ponder i % 15 equals 0 ::
```

```
    speak("FizzBuzz")
  ;; otherwise ponder i % 3 equals 0 ::
    speak("Fizz")
  ;; otherwise ponder i % 5 equals 0 ::
    speak("Buzz")
  ;; otherwise ::
    speak(i)
  ;;
;;
```

Fibonacci (Memoized)

```
forge memo be :: ;;

conjure fib(n) ::
  ponder n &lt;= 1 :: yield n ;;
  ponder contains(memo, morph(n, "text")) ::
    yield memo[morph(n, "text")]
  ;;
  forge result be fib(n - 1) + fib(n - 2)
  memo[morph(n, "text")] be result
  yield result
;;

within i be sequence(0, 30) ::
  speak("fib(" + morph(i, "text") + ") =", fib(i))
;;
```

Binary Search

```
conjure binarySearch(arr, target) ::
  forge lo be 0
  forge hi be measure(arr) - 1

  cycle lo &lt;= hi ::
    forge mid be ground((lo + hi) / 2)
    ponder arr[mid] equals target ::
```

```
      yield mid
    ;; otherwise ponder arr[mid] &lt; target ::
      lo be mid + 1
    ;; otherwise ::
      hi be mid - 1
    ;;
  ;;

  yield -1
;;

forge sorted be [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]
speak(binarySearch(sorted, 23))   // 5
speak(binarySearch(sorted, 50))   // -1
```

Quicksort

```
conjure quicksort(arr) ::
  ponder measure(arr) &lt;= 1 :: yield arr ;;

  forge pivot be arr[ground(measure(arr) / 2)]
  forge less be sift(arr, x -&gt; x &lt; pivot)
  forge equal be sift(arr, x -&gt; x equals pivot)
  forge greater be sift(arr, x -&gt; x &gt; pivot)

  yield quicksort(less) + equal + quicksort(greater)
;;

speak(quicksort([3, 6, 8, 10, 1, 2, 1]))
// [1, 1, 2, 3, 6, 8, 10]
```

Stack-Based Calculator

```
conjure calculate(expression) ::
  forge tokens be shatter(expression, " ")
  forge stack be Stack()
```

```
  within token be tokens ::
    ponder token equals "+" ::
      forge b be stack.pop()
      forge a be stack.pop()
      stack.push(a + b)
    ;; otherwise ponder token equals "-" ::
      forge b be stack.pop()
      forge a be stack.pop()
      stack.push(a - b)
    ;; otherwise ponder token equals "*" ::
      forge b be stack.pop()
      forge a be stack.pop()
      stack.push(a * b)
    ;; otherwise ponder token equals "/" ::
      forge b be stack.pop()
      forge a be stack.pop()
      stack.push(a / b)
    ;; otherwise ::
      stack.push(morph(token, "number"))
    ;;
  ;;

  yield stack.pop()
;;

speak(calculate("3 4 + 2 *"))    // (3+4)*2 = 14
speak(calculate("5 1 2 + 4 * + 3 -"))  // 5+(1+2)*4-3 = 14
```

Caesar Cipher

```
conjure caesarEncode(text, shift) ::
  forge result be ""
  within ch be chars(text) ::
    forge code be locate("ABCDEFGHIJKLMNOPQRSTUVWXYZ", upper(ch))
    ponder code &gt;= 0 ::
      forge shifted be (code + shift) % 26
      result be result + "ABCDEFGHIJKLMNOPQRSTUVWXYZ"[shifted]
    ;; otherwise ::
```

```
      result be result + ch
    ;;
  ;;
  yield result
;;

speak(caesarEncode("HELLO WORLD", 3))   // KHOOR ZRUOG
speak(caesarEncode("KHOOR ZRUOG", 23))  // HELLO WORLD (decode with
```

Word Frequency Counter

```
conjure wordFrequency(text) ::
  forge words be shatter(lower(trim(text)), " ")
  forge freq be ::;;

  within word be words ::
    forge w be replace(replace(word, ".", ""), ",", "")
    ponder measure(w) &gt; 0 ::
      ponder contains(freq, w) ::
        freq[w] be freq[w] + 1
      ;; otherwise ::
        freq[w] be 1
      ;;
    ;;
  ;;

  yield freq
;;

forge text be "the quick brown fox jumps over the lazy dog the fox
forge freq be wordFrequency(text)
speak(freq)
// Sort by frequency
forge sorted be sort(inscriptions(freq), k -&gt; -freq[k])
```

Prime Sieve

```
conjure sieve(limit) ::
  forge primes be each(sequence(2, limit + 1), x -&gt; x)
  forge result be []

  cycle measure(primes) &gt; 0 ::
    forge p be primes[0]
    gather(result, p)
    primes be sift(primes, x -&gt; x % p differs 0)
  ;;

  yield result
;;

speak(sieve(50))
// [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
```

Linked-List Graph Traversal (BFS)

```
// BFS using a Queue
conjure bfs(graph, start) ::
  forge visited be Set()
  forge queue be Queue()
  forge order be []

  queue.enqueue(start)
  visited.add(start)

  cycle isnt queue.isEmpty() ::
    forge node be queue.dequeue()
    gather(order, node)

    within neighbor be graph[node] ::
      ponder isnt visited.has(neighbor) ::
        visited.add(neighbor)
        queue.enqueue(neighbor)
      ;;
    ;;
  ;;
```

```
  yield order
;;

forge graph be ::
  "A": ["B", "C"],
  "B": ["A", "D", "E"],
  "C": ["A", "F"],
  "D": ["B"],
  "E": ["B", "F"],
  "F": ["C", "E"]
;;

speak(bfs(graph, "A"))  // ["A", "B", "C", "D", "E", "F"]
```

Turtle Mandala

```
canvas(600, 600)
clear("#000000")
turtle()
penwidth(1.5)

forge petals be 12
forge petalAngle be 360 / petals

within i be sequence(petals) ::
  pencolor(hue(i * petalAngle))
  within j be sequence(60) ::
    forward(j * 0.5)
    right(6)
  ;;
  penup()
  home()
  pendown()
  setheading(i * petalAngle)
;;
```

JavaScript Interop (JS Interpreter Only)

The browser-based interpreter includes a special `js` keyword for calling JavaScript functions directly from sdev code. This allows integration with browser APIs, DOM manipulation, and third-party JS libraries.

Three Syntax Forms

```
// 1. Single-line expression
js &lt;expression&gt;

// 2. Parenthesized form (for objects, multi-line expressions)
js (
  &lt;expression&gt;
)

// 3. Statement block
js {
  &lt;statements&gt;
}
```

Basic Examples

```
// Browser dialog
js alert("Hello from sdev!")

// Read browser properties
forge width be js window.innerWidth
speak("Width:", width)

// DOM manipulation
js document.body.style.backgroundColor = "#1a1a2e"
```

```
js document.title = "My sdev App"

// Math
forge angle be js Math.PI / 6
forge result be js Math.sin(0.5)
```

Object Literals

```
// Use parenthesized form for object literals
forge opts be js ({
  dragging: true,
  zoom: 12,
  center: [51.505, -0.09]
})

// Nested objects
forge config be js ({
  api: {
    url: "https://api.example.com",
    timeout: 5000
  },
  debug: false
})
```

Arrow Functions

```
// Map/filter in JS
forge doubled be js [1, 2, 3, 4].map(x =&gt; x * 2)
forge filtered be js [1, 2, 3, 4].filter(x =&gt; x &gt; 2)

// Event handlers
js document.getElementById("btn").addEventListener("click", (e) =&g
  console.log("Clicked!", e.target)
})

// Use sdev variables in JS arrows
forge multiplier be 5
```

```
forge result be js [1, 2, 3].map(x =&gt; x * multiplier)
```

Multi-line JS Blocks

```
js {
  const canvas = document.createElement("canvas")
  canvas.width = 400
  canvas.height = 300
  document.body.appendChild(canvas)

  const ctx = canvas.getContext("2d")
  ctx.fillStyle = "navy"
  ctx.fillRect(0, 0, 400, 300)
  ctx.fillStyle = "white"
  ctx.font = "24px monospace"
  ctx.fillText("Made in sdev!", 100, 150)
}
```

Leaflet Map Integration

```
// Initialize a Leaflet map
forge map be js L.map("map-container").setView([51.505, -0.09], 13

js L.tileLayer("https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png
  attribution: "© OpenStreetMap contributors"
})).addTo(map)

// Add markers
conjure addMarker(lat, lng, label) ::
  js L.marker([lat, lng]).addTo(map).bindPopup(label)
;;

addMarker(51.505, -0.09, "London, UK")
addMarker(48.8566, 2.3522, "Paris, France")
addMarker(52.52, 13.405, "Berlin, Germany")
```

Complete Reference Card

Keywords

| Keyword | Purpose | |---------|---------| | `forge` | Declare a variable | | `be` | Assign a value | | `conjure` | Declare a function | | `yield` | Return a value from a function | | `ponder` | `if` statement | | `otherwise` | `else` clause | | `cycle` | `while` loop | | `iterate` | `for-each` loop (iterate x through list) | | `through` | Used with iterate | | `within` | `for-in` loop (within x be list) | | `yeet` | `break` — exit loop | | `skip` | `continue` — skip iteration | | `attempt` | `try` block | | `rescue` | `catch` block | | `yep` | Boolean `true` | | `nope` | Boolean `false` | | `void` | Null / absent value | | `also` | Logical AND | | `either` | Logical OR | | `isnt` | Logical NOT | | `equals` | Equality comparison | | `differs` | Inequality comparison | | `essence` | Define a class | | `extend` | Class inheritance | | `self` | Instance reference inside class methods | | `super` | Parent class reference | | `new` | Create class instance | | `summon` | Import from Gist | | `async` | Mark function as async | | `await` | Wait for async operation | | `spawn` | Run function concurrently |

Special Symbols

| Symbol | Purpose | |--------|---------| | `::` | Start a block | | `;;` | End a block | | `->` | Lambda arrow | | `\|>` | Pipe operator | | `~` | Ternary condition operator | | `:` | Ternary else / dict key-value separator | | `//` | Line comment | | `#`

| Line comment (Python style) | | ^ | Power operator | | <> | Inequality (alternative to differs) |

Built-in Constants

Constant	Value
PI	3.141592653589793
TAU	6.283185307179586
E	2.718281828459045
INFINITY	Infinity

Quick Reference Examples

```
// ── Variables ──
forge x be 42
forge name be "Alice"
forge active be yep

// ── Arithmetic ──
speak(2 ^ 10)     // 1024
speak(10 % 3)     // 1
speak(-5 + 8)     // 3

// ── String ──
speak("Hello" + " " + "World")
speak(upper("hello"))        // HELLO
speak(measure("hello"))      // 5

// ── List ──
forge nums be [1, 2, 3, 4, 5]
gather(nums, 6)              // [1,2,3,4,5,6]
speak(nums[-1])              // 6
speak(sort(nums))            // [1,2,3,4,5,6]

// ── Conditionals ──
ponder x &gt; 0 ::
  speak("positive")
;; otherwise ::
```

```
  speak("non-positive")
;;

// ── Ternary ──
forge label be x &gt; 0 ~ "pos" : "neg"

// ── Loop ──
within i be sequence(5) :: speak(i) ;;  // 0 1 2 3 4
cycle x &gt; 0 :: x be x - 1 ;;

// ── Function ──
conjure add(a, b) :: yield a + b ;;
forge double be x -&gt; x * 2

// ── Pipe ──
[1,2,3,4,5] |&gt; each(x -&gt; x * 2) |&gt; sift(x -&gt; x &gt; 4)
// [6, 8, 10]

// ── OOP ──
essence Dog ::
  conjure init(self, name) :: self.name be name ;;
  conjure bark(self) :: speak(self.name + ": WOOF!") ;;
;;
forge rex be new Dog("Rex")
rex.bark()

// ── Error Handling ──
attempt ::
  forge n be morph("not a number", "number")
;; rescue err ::
  speak("Error:", err)
;;

// ── Higher-order ──
forge evens be sift(sequence(10), x -&gt; x % 2 equals 0)
forge sum be fold(evens, 0, (a, b) -&gt; a + b)
speak(sum)   // 20

// ── Collections ──
```

```
forge s be Set()
s.add(1) s.add(2) s.add(2)
speak(s.values())   // [1, 2]

forge q be Queue()
q.enqueue("a") q.enqueue("b")
speak(q.dequeue())   // a

// ── Matrix ──
forge a be [[1,2],[3,4]]
forge b be [[5,6],[7,8]]
speak(matmul(a, b))   // [[19,22],[43,50]]

// ── Graphics ──
canvas(400, 400)
clear("#111")
fill(hue(120))
circle(200, 200, 100)

// ── Turtle ──
turtle()
within i be sequence(4) ::
  forward(100)
  right(90)
;;
```

This documentation covers sdev version 1.x. For the latest updates and additional examples, visit the sdev IDE.